

Introduction To Machine Learning

EE-439

Project Report

**Predicting Diabetes Risk from Behavioural and
Demographic Health Indicators**



Submitted To:

Sir Mujtaba Kamal Pasha

Submitted by:

Faris Mujtaba Qureshi

2022-EE-80

Mustajib Sami

2022-EE-71

Table of Contents

1. Introduction.....	5
1.1 Project Title.....	5
1.2 Problem Statement.....	5
1.3 Objectives	5
2. Dataset.....	6
2.1 Dataset Overview.....	6
2.2 Description of the Dataset.....	6
2.3 Feature Description	7
2.4 Class Distribution.....	8
2.5 Exploratory Data Analysis	9
2.6 Why This Dataset Was Selected	12
3. Data Preprocessing and Data Cleaning.....	13
3.1 Overview of the Steps Applied	13
3.2 Duplicate Removal.....	13
3.3 Outlier Treatment on BMI	13
3.4 Target Transformation	14
3.5 Feature Scaling.....	14
3.6 Train/Test Split	14
3.7 Preventing Information Leakage.....	15
3.8 Saved Artefacts	15
3.9 Class Imbalance Strategy	15
4. Feature Extraction and Selection	15
4.1 The Nature of the Features.....	16
4.2 Measuring Feature Usefulness	16
4.3 Would Fewer Features Do?	17
4.4 Decision	18
5. Model Selection	18
5.1 Criteria Guiding the Selection.....	18
5.2 The Four Selected Models	18
5.3 Models Considered but Not Selected.....	19
6. Training.....	19
6.1 Training Strategy	19

6.2 Correcting for Class Imbalance.....	19
6.3 Model Configuration.....	20
6.4 Computational Cost	20
7. Validation and Results	20
7.1 Validation Strategy	20
7.2 Cross-Validated Performance	21
7.3 Confusion Matrices for All Four Models.....	22
7.4 Interpretation.....	23
7.5 What the Models Learned	24
7.6 Summary of Findings.....	27
8. Comparative Analysis	28
8.1 Side-by-Side Comparison	28
8.2 Best and Worst Performer.....	28
8.3 Why the Performance Differed.....	28
8.4 Strengths and Limitations of Each Model.....	29
9. Selection of the Best Model	30
9.1 The Decision	30
9.2 Reasoning.....	30
9.3 The Argument Against, and Why It Was Rejected.....	31
10. Testing: Final Evaluation on the Held-Out Set.....	31
10.1 Headline Results	31
10.2 Confusion Matrix for the Selected Model.....	32
10.3 The Decision Threshold Is a Policy Choice.....	33
10.4 Conclusion	34
11. References.....	34
Appendix A — Complete Code	35
A.1 Data Exploration and Preprocessing	35
A.2 Model Training and Selection.....	39
A.3 Final Evaluation	43

Table of Figures

Figure 1: Distribution of the three target classes. The prediabetes group is very small, at under two percent of all respondents.	8
Figure 2: Pearson correlation matrix for all 22 variables. Red indicates a positive relationship and blue a negative one.....	9
Figure 3: Correlation of each individual feature with the diabetes target, sorted from most negative to most positive.	10
Figure 4: Observed diabetes prevalence across four of the strongest risk factors.	11
Figure 5: BMI distribution by diabetes status. The curves are shifted but overlap substantially.	12
Figure 6: Cross-validated ROC-AUC as features are added in order of permutation importance.....	17
Figure 7: Cross-validated performance of the four candidate models across all five metrics.	22
Figure 8: Confusion matrices for all four models on the validation folds. Percentages are of the row, so the bottom-left cell is the proportion of diabetic people missed.	23
Figure 9: Logistic Regression odds ratios. A value above 1.0 raises the odds of diabetes, below 1.0 lowers them.	25
Figure 10: The first two levels of the Decision Tree. Each node shows the question asked, how many records reach it, and the resulting class.....	26
Figure 11: LightGBM permutation importance, measured as the drop in ROC-AUC when each feature is shuffled. Bars show the standard deviation over five repeats.	27
Figure 12: Confusion matrix for LightGBM on the held-out test set at the default decision threshold of 0.50.	32
Figure 13: Precision, recall and F1 across the full range of decision thresholds, with three candidate operating points marked.....	33

Predicting Diabetes Risk from Behavioural and Demographic Health Indicators

1. Introduction

This report documents an applied machine learning project on the prediction of diabetes risk from routine health and lifestyle information. The dataset has been obtained and studied, the data has been cleaned and prepared, four candidate models have been selected and trained, and their performance has been measured and compared. What remains is to tune the strongest of them and evaluate it once against the held-out test set.

The sections that follow describe the dataset and its characteristics, the preprocessing applied to it, how the features were analysed and selected, the reasoning behind the choice of models, how those models were trained and validated, and how the chosen model performed when finally tested. Every quantitative figure quoted in this report was produced by running the pipeline on the data rather than taken from the dataset documentation.

1.1 Project Title

Predicting Diabetes Risk from Behavioural and Demographic Health Indicators Using the CDC BRFSS 2015 Survey.

1.2 Problem Statement

Type 2 diabetes is one of the most common chronic conditions worldwide, and a large share of the people who have it do not know they have it. The condition develops slowly and its early symptoms are easy to dismiss, so a diagnosis often arrives only after damage to the eyes, kidneys, nerves or heart has already begun. Confirming a diagnosis requires a laboratory blood test such as HbA1c or fasting plasma glucose. Those tests are reliable, but they cost money, require a clinic visit, and are simply not available to a large part of the population.

This creates a practical gap. We cannot test everybody, but we would like to know who is most worth testing. The information needed to make that judgement age, weight, blood pressure history, general health, physical activity, and similar details can be collected in a few minutes without any medical equipment.

The problem addressed in this project is therefore the following: given only non-laboratory, self-reported health and lifestyle information about a person, can a machine learning model reliably identify whether that person is likely to have diabetes? This is framed as a supervised binary classification task. The aim is not to replace clinical diagnosis, but to act as a low-cost screening step that flags high-risk individuals who should then be referred for a proper blood test.

1.3 Objectives

The project sets out to achieve the following:

1. To obtain and study a large, real-world health survey dataset containing both diabetic and non-diabetic respondents.
2. To perform exploratory data analysis in order to understand the size, structure, data types and class distribution of the dataset.
3. To identify which behavioural and demographic indicators are most strongly associated with diabetes, using correlation analysis and group-wise prevalence.
4. To apply appropriate preprocessing, including duplicate removal, outlier treatment, feature scaling and a stratified train/test split.
5. To train and compare several classification models, ranging from a simple interpretable baseline to ensemble methods.
6. To evaluate those models using metrics that remain meaningful under class imbalance, with particular attention to recall on the diabetic class.
7. To draw conclusions about which features carry the most predictive signal and whether survey data alone is sufficient for practical screening.

2. Dataset

This section describes the dataset used in the project: where it comes from, what it contains, how its target variable is distributed, and why it was chosen in preference to the alternatives that were considered.

2.1 Dataset Overview

Attribute	Details
Dataset name	Diabetes Health Indicators Dataset (BRFSS 2015)
Source	Kaggle — derived from the CDC Behavioral Risk Factor Surveillance System (BRFSS) 2015
Dataset link	https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset
File used	diabetes_012_health_indicators_BRFSS2015.csv
Number of instances	253,680 respondents
Number of features	21 predictor features (22 columns including the target)
Target variable	Diabetes_012 — 0 = no diabetes, 1 = prediabetes, 2 = diabetes
Data type	CSV (structured tabular data; all columns numeric, float64)
Missing values	None (0 missing values across all 22 columns)
Duplicate rows	23,899 (9.42% of the dataset)

2.2 Description of the Dataset

Source and Origin

The Behavioural Risk Factor Surveillance System is an annual health survey conducted by the Centers for Disease Control and Prevention in the United States, collecting responses from hundreds of thousands of adults every year on their health status, chronic conditions and lifestyle behaviours.

Version Used in This Project

The version used in this project is a cleaned subset of the 2015 survey, published on Kaggle. From the several hundred questions in the original survey, 21 variables were retained that prior research has associated with diabetes risk, all encoded numerically.

Why We Chose This Dataset

Several other datasets were evaluated first, but most were rejected for one of three reasons. The available clinical datasets were too small. The largest held only around 770 records with few parameters, which would have left the model under-trained. Others were unrealistically clean and internally consistent, suggesting synthetic rather than real-world data. Several also had a 50/50 split between diabetic and non-diabetic cases, which does not reflect reality, since roughly 1 in 12 adults is diagnosed with diabetes.

Suitability of the BRFSS Subset

The BRFSS subset avoids these problems. It contains approximately 250,000 records drawn from a randomised, real-world population, and it carries the imperfections of genuine survey data, including some missing and inconsistent values that require cleaning before modelling.

2.3 Feature Description

The 21 predictor variables fall into three broad groups: medical history, lifestyle and access to healthcare, and demographics.

Feature	Type	Description
HighBP	Binary	Has been told they have high blood pressure
HighChol	Binary	Has been told they have high cholesterol
CholCheck	Binary	Cholesterol check within the past five years
BMI	Continuous	Body Mass Index (range 12 to 98)
Smoker	Binary	Has smoked at least 100 cigarettes in their life
Stroke	Binary	Has ever been told they had a stroke
HeartDiseaseorAttack	Binary	Coronary heart disease or myocardial infarction
PhysActivity	Binary	Physical activity in the past 30 days
Fruits	Binary	Consumes fruit one or more times per day
Veggies	Binary	Consumes vegetables one or more times per day
HvyAlcoholConsump	Binary	Heavy alcohol consumption
AnyHealthcare	Binary	Has any kind of health coverage
NoDocbcCost	Binary	Could not see a doctor in the past year due to cost
GenHlth	Ordinal	Self-rated general health, 1 (excellent) to 5 (poor)
MentHlth	Discrete	Days of poor mental health in the past 30 days
PhysHlth	Discrete	Days of poor physical health in the past 30 days
DiffWalk	Binary	Serious difficulty walking or climbing stairs
Sex	Binary	0 = female, 1 = male
Age	Ordinal	Age bracket, 1 (18–24) to 13 (80 or older)

Education	Ordinal	Education level, 1 (none) to 6 (college graduate)
Income	Ordinal	Income bracket, 1 (< \$10k) to 8 ($\geq$ \$75k)

2.4 Class Distribution

The target variable has three classes, and they are severely imbalanced. The exact counts are given below.

Class	Meaning	Count	Percentage
0	No diabetes	213,703	84.24%
1	Prediabetes	4,631	1.83%
2	Diabetes	35,346	13.93%
	Total	253,680	100.00%

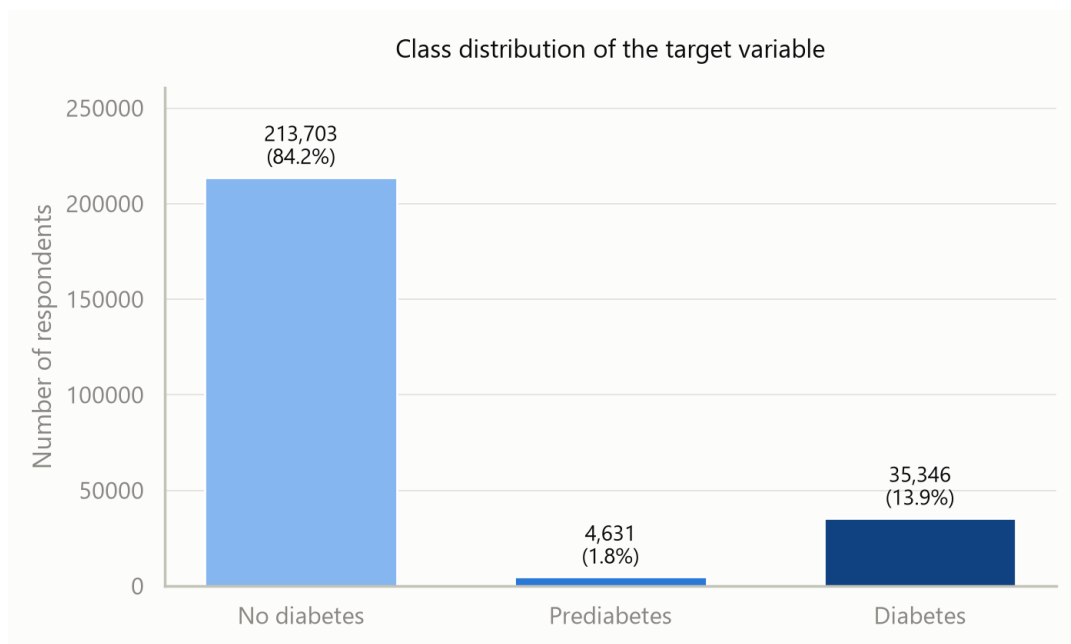


Figure 1: Distribution of the three target classes. The prediabetes group is very small, at under two percent of all respondents.

The imbalance has a direct consequence for modelling. A model that simply predicted “no diabetes” for every single respondent would achieve 84.24% accuracy while being completely useless. Accuracy alone is therefore not an acceptable measure of success on this dataset, and the prediabetes class in particular is too small for a classifier to learn reliably.

For this reason the project collapses the target into a binary variable, where classes 1 and 2 are merged into a single positive class representing any form of impaired glucose regulation. After duplicate rows are also removed, this yields 229,781 records with a positive rate of 17.29% — still imbalanced, but workable.

2.5 Exploratory Data Analysis

Because every column is numeric, a correlation matrix can be computed across all 22 variables at once. Figure 2 presents this matrix.

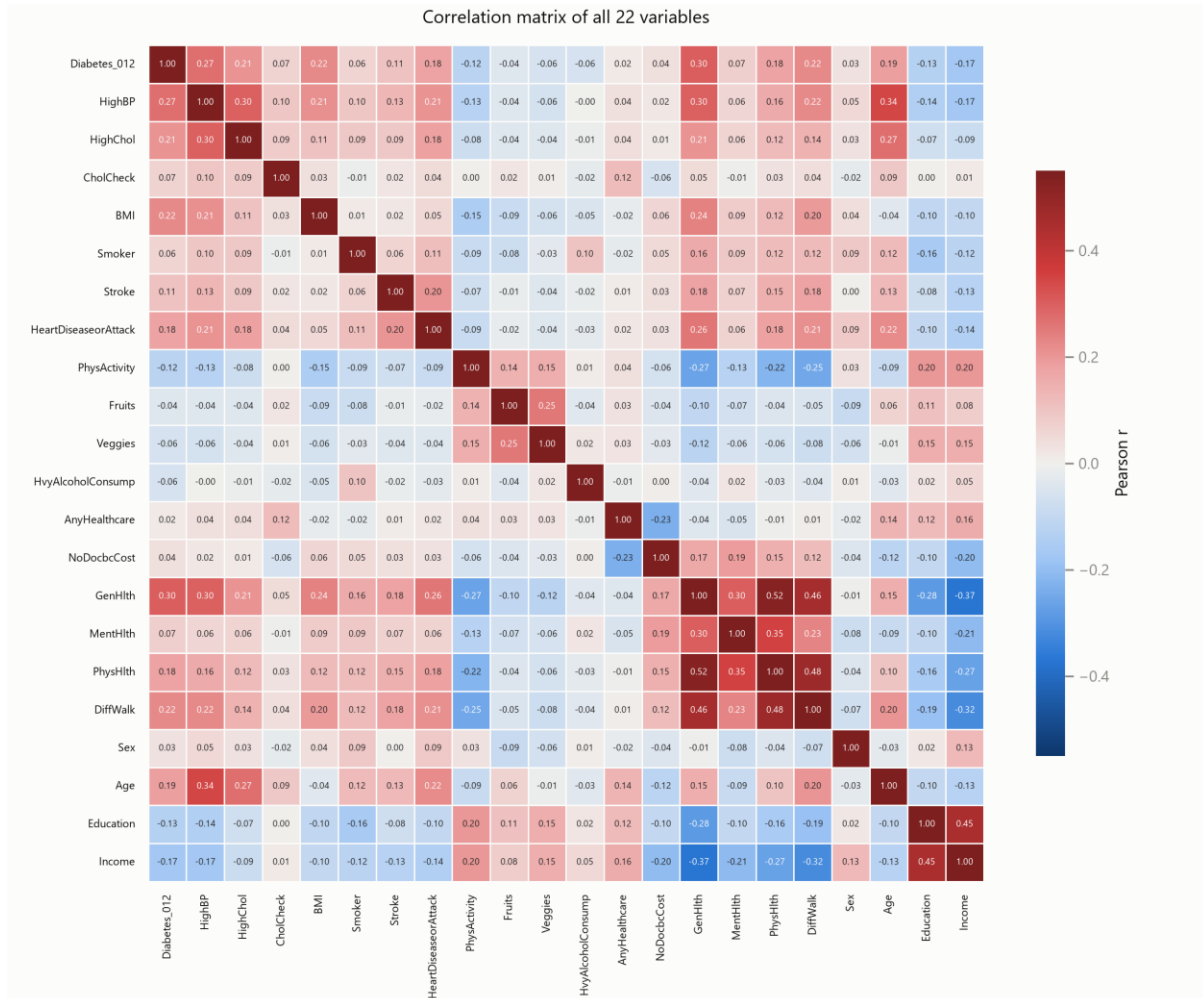


Figure 2: Pearson correlation matrix for all 22 variables. Red indicates a positive relationship and blue a negative one.

No single feature dominates. The strongest correlation with the target is general health at 0.30, followed by high blood pressure at 0.27, BMI and difficulty walking at 0.22, and high cholesterol at 0.21. On the negative side, income (-0.17), education (-0.13) and physical activity (-0.12) are all associated with lower diabetes rates. Figure 3 ranks every feature by this correlation.

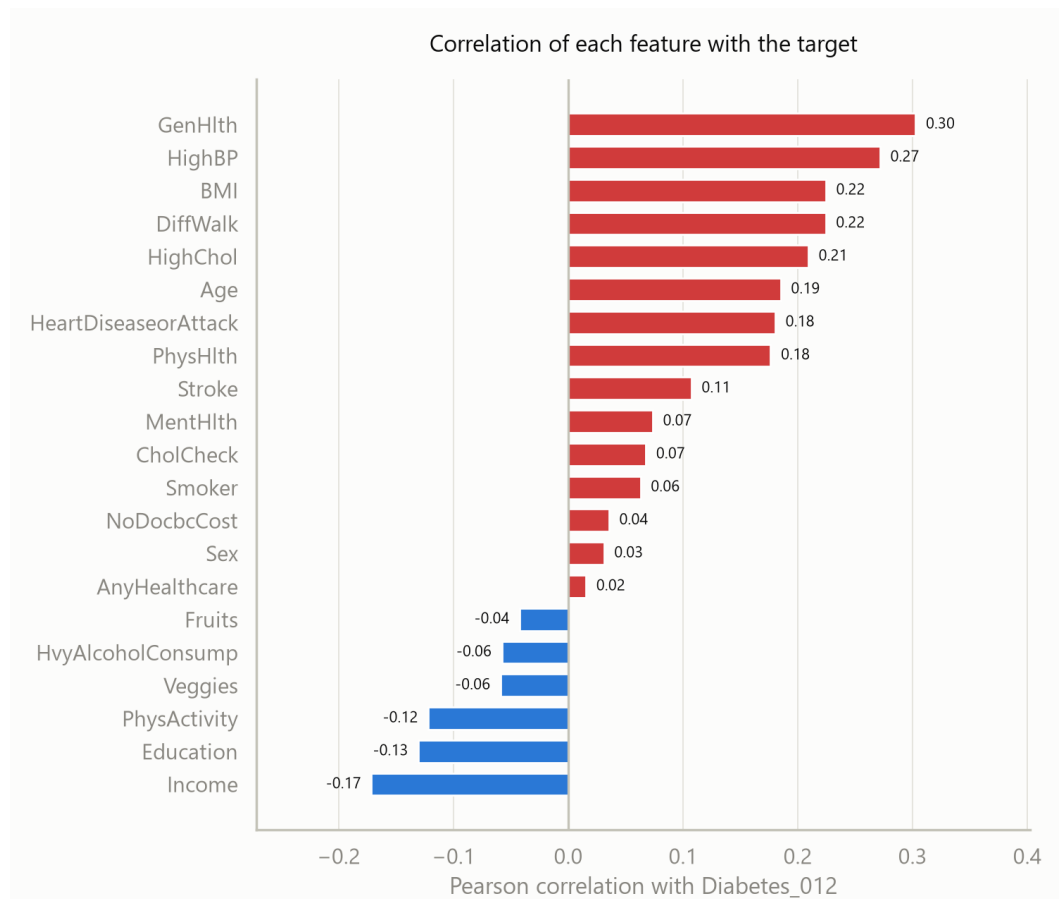


Figure 3: Correlation of each individual feature with the diabetes target, sorted from most negative to most positive.

The fact that all correlations are moderate is itself informative. It suggests that no one variable can identify diabetes on its own, and that a model combining many weak signals is likely to outperform any simple rule. It also indicates that non-linear models may have an advantage, since Pearson correlation only captures linear relationships.

Correlation coefficients are somewhat abstract, so Figure 4 shows the same relationships in more intuitive terms: for each level of a risk factor, what percentage of respondents actually reported having diabetes.

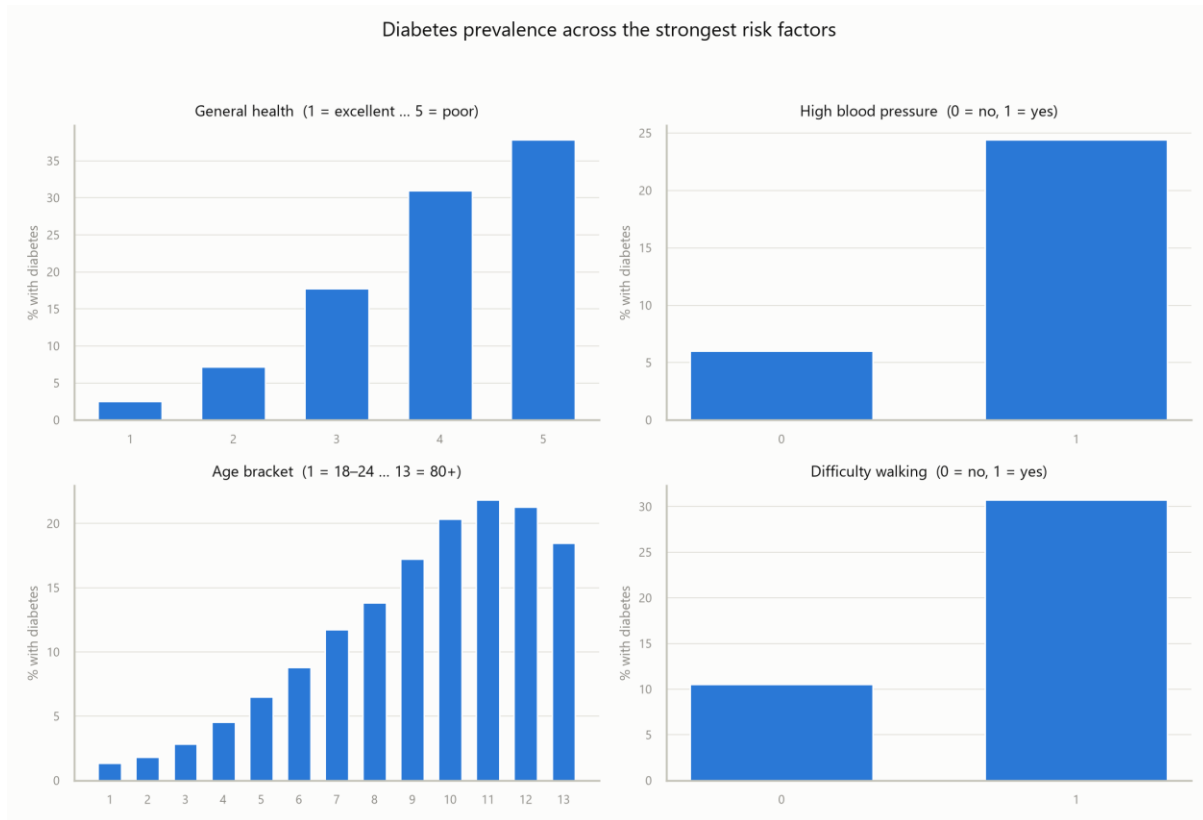


Figure 4: Observed diabetes prevalence across four of the strongest risk factors.

These patterns are clear and clinically sensible. Diabetes prevalence rises steadily from 2.5% among those in excellent general health to 37.9% among those in poor health. Respondents with high blood pressure report diabetes at 24.4% against 6.0% for those without, a fourfold difference. Prevalence climbs with age up to the 70–74 bracket and then declines slightly in the oldest groups, which is likely a survival effect. Difficulty walking is associated with a threefold increase, from 10.5% to 30.7%.

Finally, Figure 5 compares the distribution of BMI across the three classes. The median BMI is 27 for non-diabetic respondents, 30 for prediabetic and 31 for diabetic. The distributions overlap heavily, which again confirms that BMI alone is not a sufficient predictor.

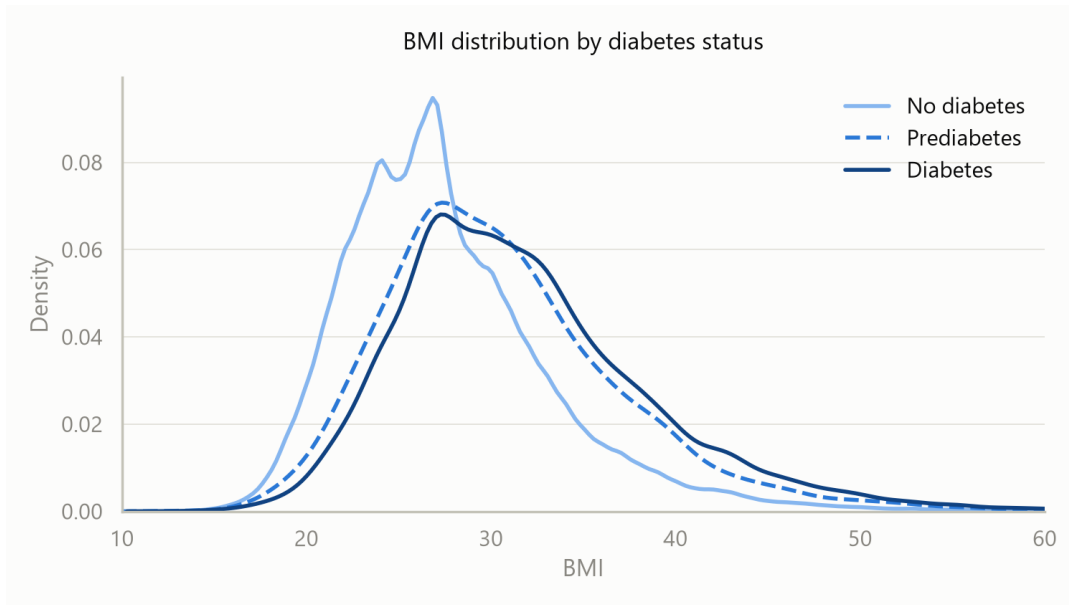


Figure 5: BMI distribution by diabetes status. The curves are shifted but overlap substantially.

2.6 Why This Dataset Was Selected

Several diabetes datasets were considered before settling on this one. The widely circulated “100,000 rows diabetes clinical dataset” on Kaggle was examined first, but it names no source institution, no collection period and no ethical approval, and its variables separate the classes unusually cleanly. It appears to be synthetically generated and was rejected for that reason. The Pima Indians dataset is genuinely clinical but contains only 768 records from a single narrow population, which limits how far any conclusion could be generalised.

The BRFSS dataset was chosen because it addresses those weaknesses:

- **It is real data.** The responses come from an actual CDC national health survey with a published methodology and codebook, so the work can be cited and defended.
- **It is large.** With 253,680 records there is more than enough data to train and validate models without worrying about sample size, and enough to hold out a substantial test set.
- **Both classes are well represented.** Even though the data is imbalanced in proportion, there are 39,977 diabetic and prediabetic respondents in absolute terms, which is a large positive class by any standard.
- **It is practically relevant.** Every feature can be collected without a laboratory, which matches the screening use case described in the problem statement.

The dataset also provides two alternative versions of the same data: a binary target file with the same 253,680 rows, and a balanced file of 70,692 rows created by undersampling the majority class to an exact fifty-fifty split. These will be useful for comparison, although the balanced version was not chosen as the primary file because its prevalence is artificial and its probability outputs would not be calibrated to any real population.

3. Data Preprocessing and Data Cleaning

Several preprocessing steps were required before the data could be used for training. This section documents each one, the reasoning behind it, and the effect it had on the data.

3.1 Overview of the Steps Applied

Seven transformations were applied to the raw file, in the order shown below. The remainder of this section explains each one, the reasoning behind it, and the effect it had on the data.

#	Step	Action taken	Effect on the data
1	Duplicate removal	Dropped exact duplicate rows	23,899 rows removed (9.42%); 253,680 → 229,781
2	Outlier treatment	Clipped BMI to the range 12–60	805 extreme values pulled to the boundary; no rows lost
3	Target transformation	Merged classes 1 and 2 into one positive class	Diabetes_012 (3 classes) → Diabetes_binary (2 classes)
4	Feature grouping	Separated columns by measurement scale	7 columns marked for scaling, 14 left unchanged
5	Train/test split	80/20 split, stratified on the target	183,824 training rows and 45,957 testing rows
6	Standardisation	StandardScaler fitted on the training set only	7 columns rescaled to zero mean and unit variance
7	Persistence	Saved processed arrays and the fitted transformer	Two artefact files written to disk

3.2 Duplicate Removal

The raw file contained 23,899 rows that were exact duplicates of another row, amounting to 9.42% of the dataset. Because most of the 21 features are binary and the remainder are coarse brackets, the number of distinct response combinations is limited, so two unrelated respondents can genuinely produce identical records. These rows were therefore not necessarily errors.

They were nevertheless removed, and this was done first, before any other transformation. The reason is methodological: if a duplicated record is assigned to the training set and its twin to the test set, the model is evaluated on a row it has already memorised, and the reported score becomes optimistic. Removing duplicates before splitting eliminates that possibility. After this step, 229,781 records remained.

3.3 Outlier Treatment on BMI

BMI was the only continuous measurement in the dataset and the only column with a wide numeric range, running from 12 to 98 in the raw file. Values above roughly 60 are physiologically very rare and, in a self-reported telephone survey, are more plausibly the result of a misheard or mis-entered figure than a genuine measurement. In the deduplicated data, 805 records fell outside the range 12 to 60.

Rather than delete those records and lose the twenty other valid answers each one contains, BMI was clipped so that any value below 12 was raised to 12 and any value above 60 was reduced to 60. Every

row was preserved. After clipping, BMI in the training set has a mean of 28.64 and a standard deviation of 6.44.

3.4 Target Transformation

As explained in Section 2.4, the three-class target was collapsed to binary by merging prediabetes and diabetes into a single positive class. The alternative of keeping all three classes was considered but rejected, because at 1.83% the prediabetes class is too small for a classifier to separate reliably.

Applying this transformation to the deduplicated data produces 39,726 positive cases against 190,055 negative cases, a positive rate of 17.29%. The dataset remains imbalanced after the change, and that imbalance is handled at training time rather than by altering the data further.

3.5 Feature Scaling

The 21 predictors do not share a common scale. Fourteen are binary indicators confined to 0 and 1, while the remaining seven span much wider ranges: BMI reaches 60, the two health-day counts run to 30, and the ordinal brackets for age, education and income run to 13, 6 and 8 respectively. Left untreated, a distance-based or gradient-based algorithm would treat BMI as far more important than any binary feature purely because its numbers are larger.

The seven wider-ranging columns were therefore standardised to zero mean and unit variance, while the fourteen binary columns were passed through unchanged. The transformation was implemented as a scikit-learn ColumnTransformer so that both groups are handled by a single reusable object.

Treatment	Columns	Count
Standardised	BMI, MentHlth, PhysHlth, GenHlth, Age, Education, Income	7
Passed through unchanged	HighBP, HighChol, CholCheck, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, DiffWalk, Sex	14

Verification on the training set confirms the seven scaled columns now have a mean of 0.0000 and a standard deviation of 1.0000.

3.6 Train/Test Split

The prepared dataset was successfully split into a training set and a testing set using an 80/20 ratio. The split was stratified on the target variable, which forces both sides to preserve the same proportion of diabetic cases as the full dataset. Without stratification, random chance could leave the test set with a noticeably different positive rate and make the evaluation unrepresentative.

A fixed random seed of 42 was used so that the same partition can be reproduced exactly on any subsequent run. The resulting subsets are as follows.

Subset	Records	Share	Non-diabetic	Diabetic	Positive rate
Training set	183,824	80%	152,043	31,781	17.29%
Testing set	45,957	20%	38,012	7,945	17.29%
Total	229,781	100%	190,055	39,726	17.29%

The positive rate is identical to two decimal places across the training set, the testing set and the dataset as a whole, which confirms that the stratification worked as intended. After transformation the training matrix has dimensions $183,824 \times 21$ and the testing matrix $45,957 \times 21$, the 21 columns in both cases being the seven standardised features followed by the fourteen binary features.

3.7 Preventing Information Leakage

One detail of the ordering above is worth drawing attention to, because getting it wrong is a common and largely invisible mistake. The scaler was fitted on the training set alone and then applied to the testing set, rather than being fitted on the full dataset before splitting.

Had the scaler been fitted first, the mean and standard deviation used to transform the training data would have been computed partly from the test rows. Information about the test set would then have influenced the training data, and the final evaluation would no longer represent genuinely unseen data. Fitting after the split keeps the test set isolated until the models are evaluated. For the same reason, any oversampling technique trialled later will be applied to the training fold only.

3.8 Saved Artefacts

So that model training can begin without repeating any of the steps above, the processed data and the fitted transformer were written to disk.

File	Contents
brfss_train_test.npz	The processed training and testing matrices, their corresponding target vectors, and the ordered list of feature names.
brfss_preprocessor.joblib	The fitted ColumnTransformer, so that any new record can be transformed in exactly the same way the models were trained on.

Preserving the fitted transformer matters for any practical use of the model. A prediction for a new respondent is only valid if their BMI, age bracket and other continuous values are scaled using the same means and standard deviations learned during training; re-fitting a fresh scaler on new data would produce silently incorrect inputs.

3.9 Class Imbalance Strategy

With a positive rate of 17.29%, the imbalance must be addressed during training rather than by resampling the whole dataset. The intended approach is to use class weighting, setting `class_weight` to `balanced` for models that support it and `scale_pos_weight` for gradient boosting. If oversampling techniques such as SMOTE are trialled, they will be applied to the training fold only, never to the test set.

4. Feature Extraction and Selection

Between cleaning the data and choosing a model sits the question of which variables the model should actually be given. This section records what was done about that: what kind of features the dataset provides, how their individual usefulness was measured, whether a smaller subset would serve just as well, and what was finally decided.

4.1 The Nature of the Features

It is worth being precise about what feature extraction means for a dataset of this kind, because the term is often associated with images, audio or free text, where numerical features must be derived from a raw signal before any model can be applied. No such step is required here. The BRFSS variables are already discrete, meaningful quantities: each one is a direct answer to a survey question, coded numerically by the CDC before publication. There is no raw signal from which features must be constructed.

What the data does require is the set of decisions described below and in Section 3: deciding how each variable should be treated according to its measurement scale, measuring how much each contributes, and deciding whether all of them should be kept. The 21 features divide by scale into three groups.

Group	Count	Members	Treatment
Binary indicators	14	HighBP, HighChol, CholCheck, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, DiffWalk, Sex	Left unchanged; already on a 0/1 scale
Ordered categories	4	GenHlth (1–5), Age (1–13), Education (1–6), Income (1–8)	Standardised; treated as ordered numbers rather than unordered labels
Counts and measurements	3	BMI, MentHlth (0–30), PhysHlth (0–30)	Standardised to zero mean and unit variance

The middle group carries an assumption worth stating plainly. Treating education or income brackets as numbers asserts that the step from one bracket to the next is comparable in size at every point on the scale, which is not strictly true. The alternative is one-hot encoding, which makes no such assumption but replaces four columns with roughly thirty-two and discards the ordering entirely. Since the ordering is genuinely informative — higher income really does correspond to lower diabetes prevalence throughout the range — the ordered treatment was kept.

No new features were derived from existing ones. Constructing interaction terms was considered and rejected: three of the four models chosen in Section 5 discover interactions on their own, so supplying them by hand would benefit only Logistic Regression while making every model harder to interpret.

4.2 Measuring Feature Usefulness

Three independent methods were used to judge how much each feature contributes, in preference to relying on any single one.

- **Correlation with the target.** Reported in Section 2.5. It is the simplest measure but captures only linear relationships, so it can understate a feature that matters in combination with others.
- **Permutation importance.** Reported in Section 7.5. Each feature is shuffled in turn and the resulting fall in ROC-AUC is recorded, which measures what the trained model actually relies on.
- **Position in the decision tree.** Also in Section 7.5. Which questions the tree chooses to ask first is a direct, readable statement of which features separate the classes most sharply.

All three agree on the leading group: self-rated general health, BMI, age, high blood pressure and high cholesterol. Agreement between three methods that work in quite different ways is considerably stronger evidence than any one of them alone would provide.

4.3 Would Fewer Features Do?

Since a handful of features dominate, it is worth asking whether the rest earn their place. This was tested directly. Features were ranked by permutation importance, and LightGBM was retrained using only the top three, five, seven, ten, fifteen and finally all twenty-one, each evaluated by the same five-fold cross-validation used everywhere else in this report.

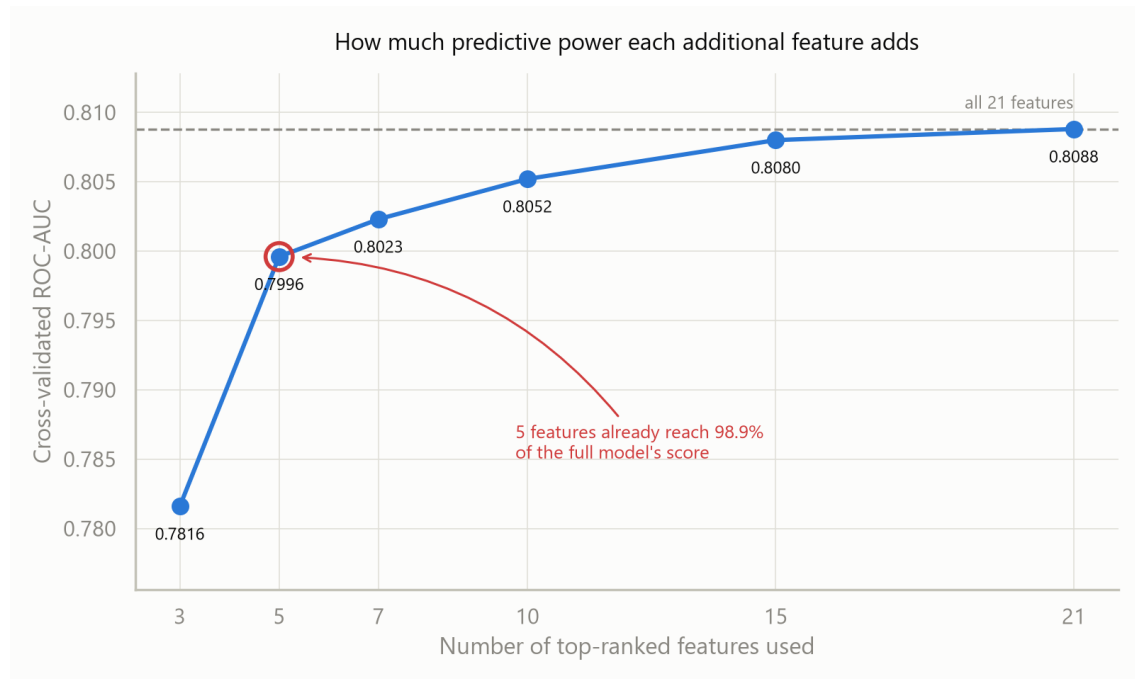


Figure 6: Cross-validated ROC-AUC as features are added in order of permutation importance.

Features used	Which ones	ROC-AUC	Recall	Loss vs all 21
3	GenHlth, BMI, Age	0.7816	0.7534	−0.0272
5	+ HighBP, HighChol	0.7996	0.7660	−0.0092
7	+ Income, CholCheck	0.8023	0.7689	−0.0065
10	+ PhysHlth, MentHlth, Sex	0.8052	0.7729	−0.0036
15	+ five more	0.8080	0.7716	−0.0008
21	all features	0.8088	0.7715	0.0000

The curve flattens quickly. Five features reach a ROC-AUC of 0.7996, which is 98.9% of what all twenty-one achieve, and ten features come within 0.0036. Everything beyond the first ten contributes almost nothing measurable.

This is a practically useful result rather than a merely technical one. It means a screening questionnaire could be cut from twenty-one questions to about ten with almost no loss of accuracy, which matters a great deal if the instrument is ever to be administered by hand or over the telephone.

It also confirms that the leading features identified in Section 4.2 are genuinely carrying the signal and not merely correlated with whatever is.

4.4 Decision

All twenty-one features were retained for the modelling that follows. Three reasons support that choice. The full set scores highest, if only slightly. None of the features is expensive to collect, since every one is a single survey question, so there is no cost argument for discarding any. And with 183,824 training records against 21 features there is no risk of the high-dimensionality problems that would justify aggressive feature reduction on a smaller dataset.

The reduced ten-feature configuration is recorded here as the preferred starting point should a shorter questionnaire ever be required, and the experiment above quantifies exactly what that shortening would cost.

5. Model Selection

Four models were selected for comparison: three simple models and one advanced one. The three simple models were deliberately drawn from three different algorithm families rather than being three variations on a single idea. Chosen that way, the comparison reveals something about the shape of the data itself, not merely which settings of one algorithm happen to work best.

5.1 Criteria Guiding the Selection

Four considerations shaped which algorithms were chosen.

- **Suitability for tabular data.** The dataset is a structured table of 21 mostly categorical features. Tree-based and linear methods are well established on data of this shape, whereas architectures designed for images or sequences offer no advantage here.
- **Tolerance of class imbalance.** With a positive rate of 17.29%, each candidate had to either support class weighting directly or allow its decision threshold to be adjusted after training.
- **Interpretability.** The intended application is medical screening, where a clinician needs to understand why a person was flagged. A model whose reasoning can be inspected is preferable to a marginally more accurate one that cannot.
- **A spread of complexity.** Including both simple and complex algorithms makes it possible to judge whether the additional complexity actually earns its place.

5.2 The Four Selected Models

Model	Family	Role in the comparison
Logistic Regression	Linear	The interpretable baseline. Each coefficient converts to an odds ratio, so the model can state how much each risk factor raises the odds of diabetes.
Decision Tree	Rule-based	Captures non-linear effects and interactions a linear model cannot, and produces a flowchart of yes/no questions that needs no statistical training to read.
Gaussian Naive Bayes	Probabilistic	The floor against which the others are judged. If a complex model cannot clearly beat it, that complexity is not earning its place.

LightGBM	Gradient-boosted ensemble	The advanced model. Gradient boosting over decision trees is the strongest general method for structured tabular data, and it corrects for imbalance directly through its <code>scale_pos_weight</code> parameter.
----------	---------------------------	--

5.3 Models Considered but Not Selected

Two further algorithms were evaluated as candidates and deliberately excluded. Recording why is as much a part of model selection as recording what was chosen.

- **K-Nearest Neighbours.** This algorithm has no real training phase; all of its work happens at prediction time. Classifying the test set would mean measuring the distance from each of 45,957 records to all 183,824 training records, which is prohibitively slow at this scale. It is also weakened by having 21 dimensions, across which distances between points become less meaningful.
- **Random Forest.** A capable model, but it belongs to the same tree-ensemble family as LightGBM, which represents that family in a stronger form. Including both would have tested the same idea twice instead of adding a genuinely different one.

6. Training

6.1 Training Strategy

All four models were trained on the 183,824-record training set prepared in Section 3. They were compared using five-fold stratified cross-validation carried out entirely within that training set: the training data is divided into five parts, each model is trained on four of them and scored on the fifth, and the process repeats five times so that every record is used for scoring exactly once.

This matters because it gives an honest estimate of performance on unseen data without consuming the test set. The 45,957 held-out records were not touched at any point during training or comparison. They remain sealed for the final evaluation, so that the figure eventually reported for the chosen model is measured on data that played no part in selecting it.

The folds are stratified, meaning each one preserves the same 17.29% positive rate as the training set as a whole. Without stratification, an unlucky division could produce folds with noticeably different class balances and make the scores unstable.

6.2 Correcting for Class Imbalance

Left uncorrected, every model would learn that predicting “not diabetic” for everybody is a reliable way to be right most of the time. Each model was therefore told to treat the minority class as more important, by a factor of roughly 4.78, which is the ratio of 152,043 non-diabetic to 31,781 diabetic records in the training set.

The mechanism differs between algorithms. Logistic Regression, the Decision Tree and LightGBM all accept a weighting parameter directly. Gaussian Naive Bayes has no such parameter, so equal prior probabilities were specified instead, which achieves the same effect for that algorithm.

6.3 Model Configuration

The settings below are sensible defaults rather than tuned values. Tuning is deliberately deferred: the purpose at this stage is a fair comparison between model families, and tuning one candidate more thoroughly than another would distort exactly the comparison being made.

Model	Configuration	Imbalance correction
Logistic Regression	max_iter = 1000, L2 regularisation (default)	class_weight = balanced
Decision Tree	max_depth = 6, min_samples_leaf = 50	class_weight = balanced
Gaussian Naive Bayes	no tuning parameters	priors = [0.5, 0.5]
LightGBM	n_estimators = 400, learning_rate = 0.05, num_leaves = 31	scale_pos_weight = 4.784

Two of these deserve a note. The Decision Tree was capped at a depth of six because an unconstrained tree on 183,824 records grows until it has effectively memorised them, producing a model that performs superbly on training data and poorly on anything else; requiring at least 50 records behind every leaf serves the same purpose. LightGBM was given a deliberately small learning rate spread over a larger number of boosting rounds, which is the standard trade in gradient boosting: many small corrections generalise better than a few large ones.

6.4 Computational Cost

Training was inexpensive. A single fit took 0.10 seconds for Gaussian Naive Bayes, 0.31 seconds for the Decision Tree, 0.46 seconds for Logistic Regression and 1.41 seconds for LightGBM. The complete cross-validated comparison of all four models, twenty fits in total, finished in under six seconds. Cost is therefore not a factor in choosing between these candidates, and the entire comparison can be re-run from scratch whenever the preprocessing changes.

7. Validation and Results

This section reports how the four trained models were validated and how they compared. Validation here is kept strictly separate from testing: the models are assessed on data held out from their own training, but that data still comes from the training set. The genuine test set remains untouched until Section 9.

7.1 Validation Strategy

The project uses the standard three-way discipline, with the second and third roles kept firmly apart.

Role	Records	Where it comes from	What it is used for
Training	about 147,000 per fold	Four fifths of the training set, per fold	Fitting the model's parameters
Validation	about 37,000 per fold	The remaining fifth, rotated five times	Comparing models, and choosing the decision threshold
Test	45,957	Held out at the very start, in Section 3.6	One final measurement, after all decisions are made

Five-fold cross-validation is used rather than a single fixed validation split because it uses the training data more efficiently: every record serves for validation exactly once, and the five resulting scores also indicate how stable each model is. The folds are stratified so that each preserves the 17.29% positive rate of the whole.

Every number in Sections 7.2 to 7.6 is a validation score obtained this way. Nothing in this section involves the test set, and no model was adjusted on the strength of a test-set result.

7.2 Cross-Validated Performance

The table below reports the mean score across the five folds for each model, ordered by ROC-AUC. Accuracy is included as required, but it must be read alongside the other columns rather than on its own, for the reason given immediately below.

Model	Accuracy	Balanced Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
LightGBM	0.708	0.733	0.346	0.772	0.478	0.809	0.460
Logistic Regression	0.715	0.729	0.349	0.749	0.476	0.802	0.440
Decision Tree	0.708	0.719	0.341	0.735	0.465	0.790	0.418
Gaussian Naive Bayes	0.697	0.702	0.327	0.709	0.448	0.762	0.374

A reader looking only at the accuracy column would conclude that all four models are poor, and would be wrong. Because 82.71% of the training records are non-diabetic, a model that simply answered “no diabetes” to every respondent would score 82.71% accuracy while identifying not a single diabetic person. Every model here scores lower than that, between 69.7% and 71.5%, and this is the direct and intended consequence of the class weighting described in Section 6.2: the models were told to stop ignoring the minority class, and they gave up raw accuracy in exchange for finding roughly three quarters of the diabetic cases instead of none.

Balanced accuracy, which averages the accuracy on each class separately rather than pooling them, is the fairer version of the same measure and is reported alongside it. On that measure the ordering matches every other metric. This distinction also has a practical consequence: Logistic Regression records the highest raw accuracy of the four at 0.715, so a selection made on accuracy alone would have chosen the wrong model.

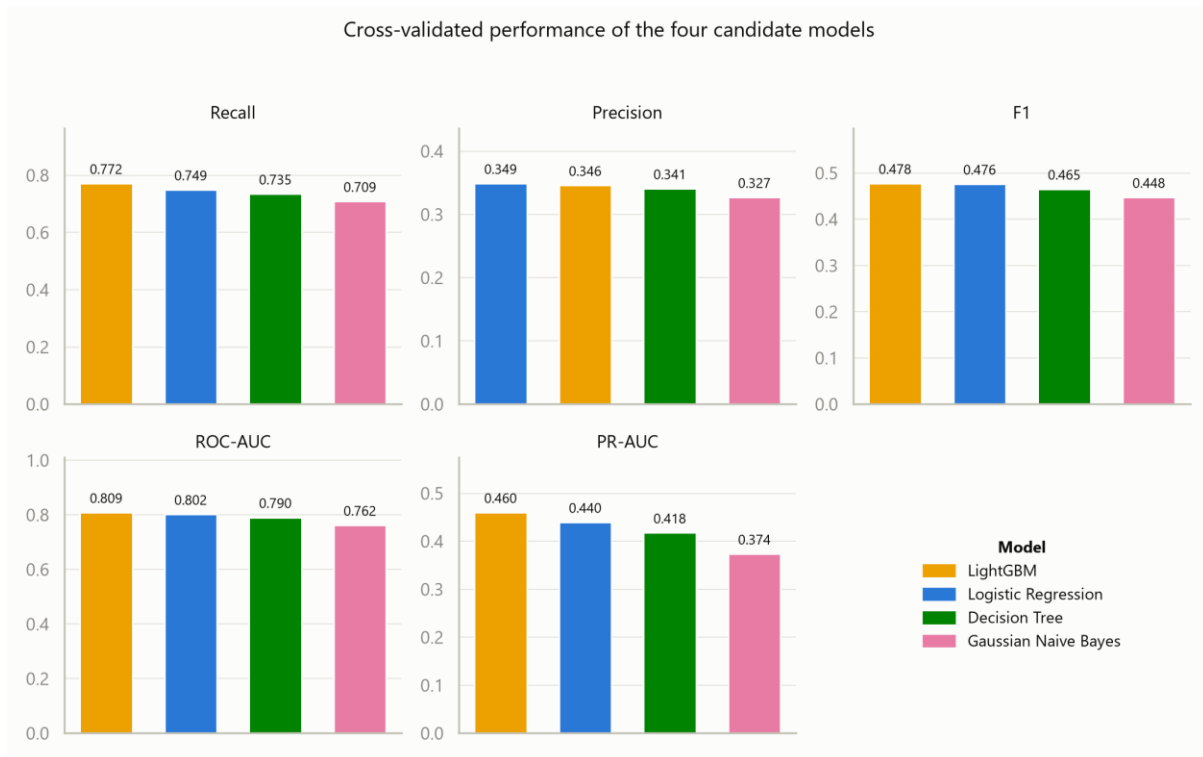


Figure 7: Cross-validated performance of the four candidate models across all five metrics.

7.3 Confusion Matrices for All Four Models

The table above summarises performance as scores. A confusion matrix shows the same performance as counts of people, which makes the practical difference between the models much easier to see. The four matrices below were produced from the validation folds, so every one of the 183,824 training records appears exactly once, predicted by a copy of the model that had not seen it.



Figure 8: Confusion matrices for all four models on the validation folds. Percentages are of the row, so the bottom-left cell is the proportion of diabetic people missed.

Model	Correctly found (TP)	Missed (FN)	% of diabetics missed	False alarms (FP)
LightGBM	24,520	7,261	22.8%	46,381
Logistic Regression	23,814	7,967	25.1%	44,421
Decision Tree	23,349	8,432	26.5%	45,272
Gaussian Naive Bayes	22,528	9,253	29.1%	46,370

Expressed this way the ranking carries more weight than the difference in ROC-AUC suggested. Against Gaussian Naive Bayes, LightGBM finds 1,992 more diabetic people among the same 31,781; against Logistic Regression it finds 706 more. Its false-alarm count is essentially the same as the others, so the additional cases are not being bought by flagging more people indiscriminately — the model is simply ranking them better.

7.4 Interpretation

The ordering is consistent across every metric: LightGBM first, Logistic Regression second, the Decision Tree third and Gaussian Naive Bayes last. That the ranking does not change depending on

which metric is consulted is itself reassuring, since it means the conclusion does not depend on the choice of measure.

The more interesting result is how small the margin is. LightGBM leads Logistic Regression by 0.0067 in ROC-AUC, a difference of less than one percent. An advanced gradient-boosted ensemble of 400 trees is therefore barely outperforming a linear model that can be written out as a single equation. In a medical screening context this is a point in favour of the simple model rather than a disappointment: where a clinician must be able to justify why a patient was flagged, a model whose reasoning is transparent has real practical value that a fractional gain in AUC does not offset.

Gaussian Naive Bayes finishing last is expected and confirms that it is doing its job as a floor. The algorithm assumes every feature is independent of the others given the class, and that assumption is plainly violated here, as Section 2.5 showed: general health correlates with poor physical health days at 0.52, and difficulty walking with the same variable at 0.48. That the other three models beat it confirms they are genuinely exploiting relationships between features rather than merely adding them up.

Precision sits between 0.327 and 0.349 for all four models, which deserves to be stated plainly rather than left inside a table. It means roughly two out of every three people the model flags do not in fact have diabetes. For a screening tool whose false alarms lead only to an inexpensive blood test this is an acceptable trade against recall of roughly 0.77, but it would not be acceptable if a positive result triggered anything costly or invasive.

7.5 What the Models Learned

A score says which model performs best but nothing about why it decides as it does. Because two of the four candidates are directly interpretable, their reasoning can be examined, and this turns out to be the most informative part of the comparison.

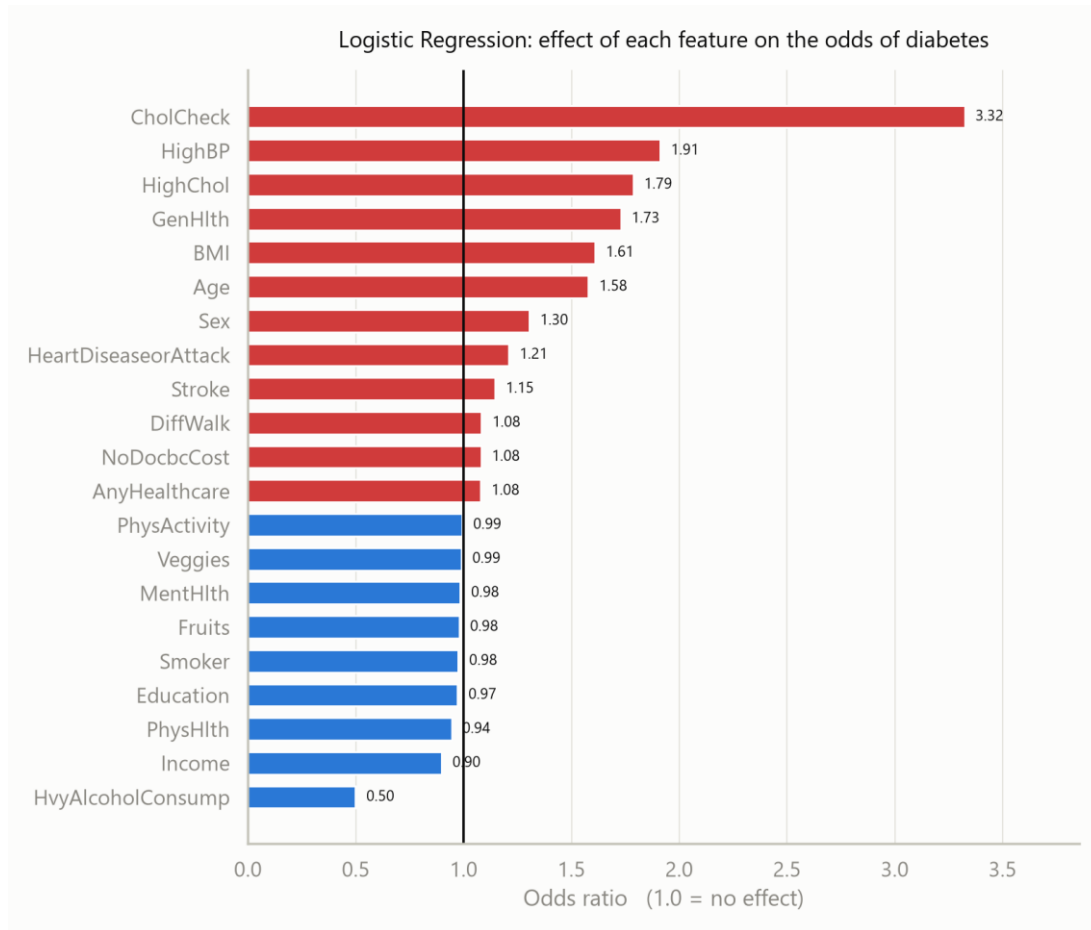


Figure 9: Logistic Regression odds ratios. A value above 1.0 raises the odds of diabetes, below 1.0 lowers them.

Most of the ranking is clinically sensible. High blood pressure nearly doubles the odds at 1.91, high cholesterol raises them by 1.79, and poorer self-rated general health, higher BMI and greater age all follow at between 1.58 and 1.73. Two coefficients, however, should not be read as causes, and identifying why is part of interpreting the model honestly.

- **Cholesterol check (odds ratio 3.32).** The largest coefficient in the model belongs to whether the respondent has had a cholesterol test in the past five years. A blood test plainly does not cause diabetes. What this captures is access to healthcare: people who regularly see a doctor are the people who get diagnosed. The model is partly detecting who has been tested rather than who is ill.
- **Heavy alcohol consumption (odds ratio 0.50).** Taken at face value this says heavy drinking halves the risk of diabetes, which is not credible. The likeliest explanation is reverse causation: people already diagnosed with diabetes are advised to reduce their alcohol intake, so the diagnosis changes the behaviour rather than the behaviour preventing the diagnosis.

Both illustrate a limitation inherent in the data rather than a fault in the model. The survey is cross-sectional, recording everything at a single point in time, so it cannot distinguish a cause from a consequence. A model built on it can predict well while still being a poor guide to what would happen if any of these factors were changed.

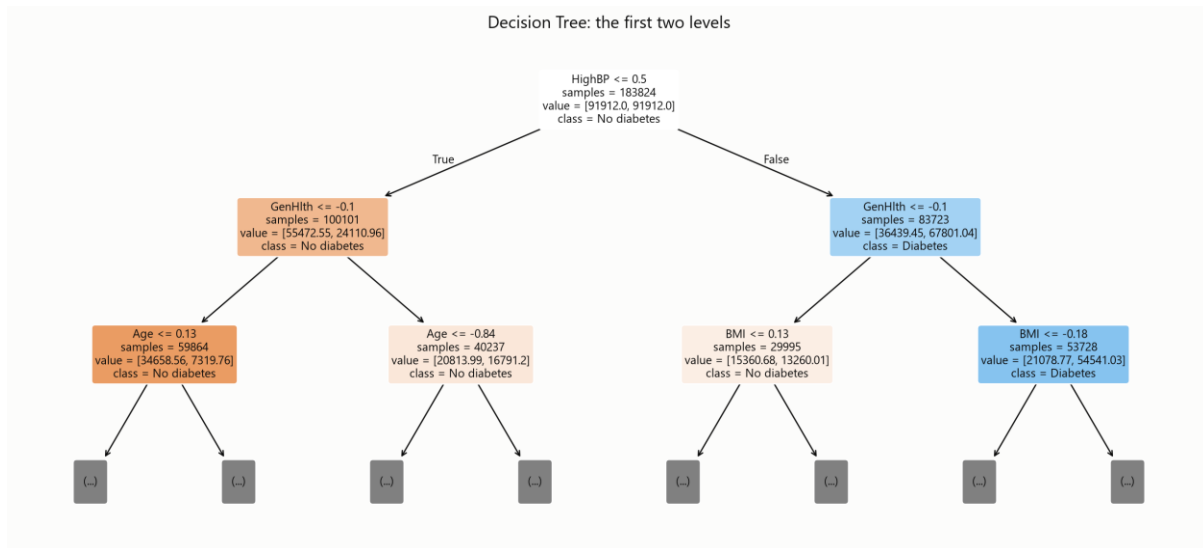


Figure 10: The first two levels of the Decision Tree. Each node shows the question asked, how many records reach it, and the resulting class.

The Decision Tree offers the same information in a more immediately readable form. Its very first question is whether the respondent has high blood pressure, which alone divides the 183,824 records into two groups with markedly different diabetes rates. Both branches then ask about self-rated general health, after which the tree turns to age on one side and BMI on the other. This is a screening procedure a clinician could follow on paper, and it agrees closely with the correlation analysis in Section 2.5.

For LightGBM, which is an ensemble of 400 trees and cannot be drawn, feature importance must be measured instead. The measure used here is permutation importance: each feature in turn is randomly shuffled and the resulting drop in ROC-AUC is recorded. A feature the model genuinely relies on causes a large drop when scrambled.

This method was chosen in preference to LightGBM's built-in importance score, which counts how often each feature is used to split the data. That count is systematically biased towards continuous variables, because a variable such as BMI offers hundreds of possible split points while a yes/no variable such as high blood pressure offers only one. Ranking by split count placed BMI first and high blood pressure eleventh, which contradicts both the correlation analysis and the Decision Tree; the permutation measure does not.

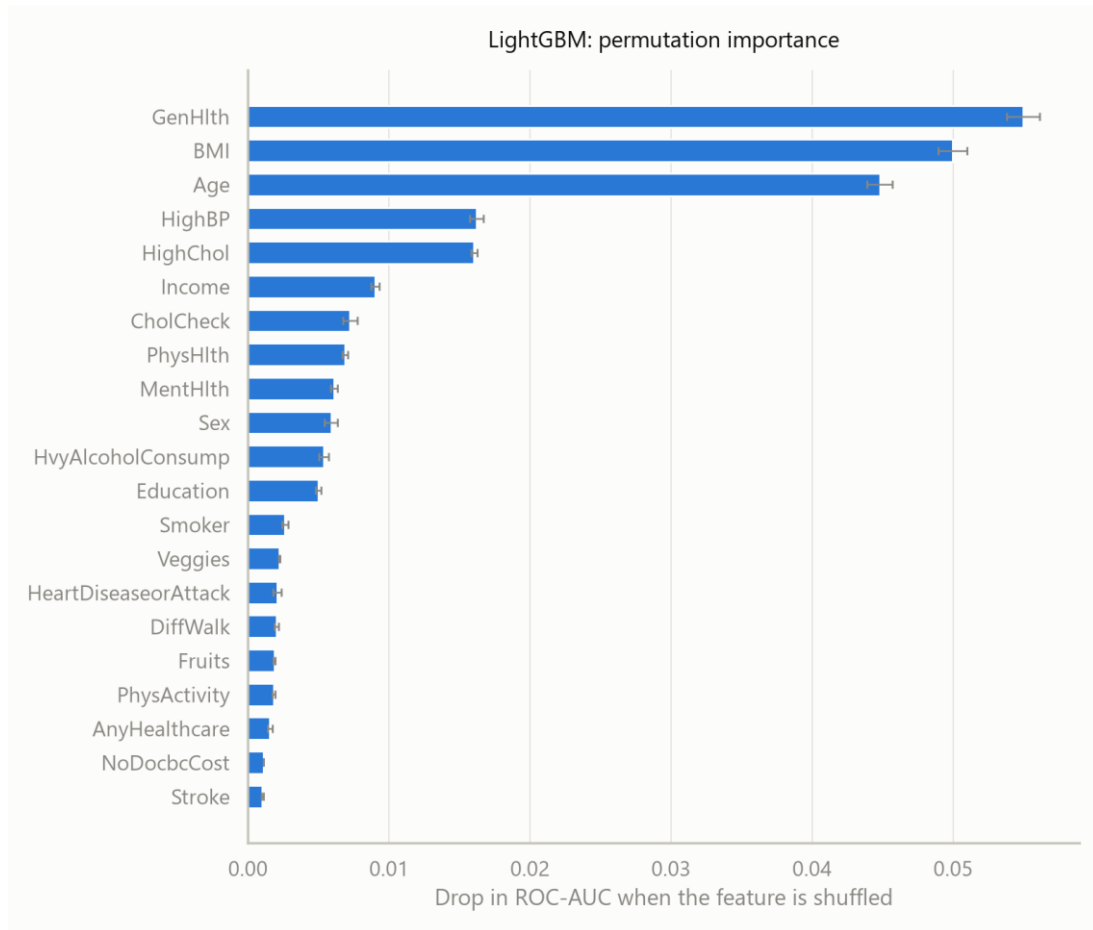


Figure 11: LightGBM permutation importance, measured as the drop in ROC-AUC when each feature is shuffled. Bars show the standard deviation over five repeats.

Three features dominate: self-rated general health at 0.055, BMI at 0.050 and age at 0.045, each roughly three times more influential than the next group of high blood pressure and high cholesterol at around 0.016. This ordering agrees with the correlation analysis of Section 2.5, where general health, blood pressure, BMI and difficulty walking were the strongest individual correlates, and it agrees with the questions the Decision Tree chose to ask first. Three independent methods converging on the same handful of variables is stronger evidence than any one of them alone.

7.6 Summary of Findings

Four conclusions follow from the results above.

- **Survey data alone carries real predictive signal.** A ROC-AUC of 0.809 from questions requiring no laboratory test and no medical equipment is a genuinely useful result for a screening tool.
- **The advanced model's advantage is marginal.** LightGBM beats Logistic Regression by less than one percent of ROC-AUC, which makes the interpretable model a serious candidate for deployment in its own right.
- **A small number of features carry most of the signal.** General health, BMI, age, blood pressure and cholesterol account for the great majority of the models' discriminating power.

- **Predictive strength is not causal evidence.** The cholesterol-check and alcohol coefficients show that some of the model's power comes from associations that would not survive being acted upon.

8. Comparative Analysis

Section 7 reported each model's scores. This section compares them directly: which performed best, which performed worst, why the four differed, and what each one is good and bad at. Every figure quoted comes from the same five-fold cross-validation on the training set, so the comparison is like for like.

8.1 Side-by-Side Comparison

The four models are ranked below on every metric. The rank column counts how many of the seven metrics each model leads on.

Rank	Model	Accuracy	Bal. Acc	Precision	Recall	F1	ROC-AUC	PR-AUC	Metrics led
1	LightGBM	0.708	0.733	0.346	0.772	0.478	0.809	0.460	5 of 7
2	Logistic Regression	0.715	0.729	0.349	0.749	0.476	0.802	0.440	2 of 7
3	Decision Tree	0.708	0.719	0.341	0.735	0.465	0.790	0.418	0 of 7
4	Gaussian Naive Bayes	0.697	0.702	0.327	0.709	0.448	0.762	0.374	0 of 7

The two metrics Logistic Regression leads on are raw accuracy and precision, by 0.007 and 0.003 respectively. As Section 7.2 explained, raw accuracy is the one measure that is actively misleading on imbalanced data, so that particular lead should not be given weight.

8.2 Best and Worst Performer

	Model	Evidence
Best	LightGBM	Highest recall (0.772), F1 (0.478), ROC-AUC (0.809), PR-AUC (0.460) and balanced accuracy (0.733). Also the most stable, with the smallest variation in recall across folds (SD 0.0045).
Worst	Gaussian Naive Bayes	Lowest on all seven metrics. Its PR-AUC of 0.374 is 0.086 below LightGBM, the largest gap between any two models in the comparison.

The gap between first and last is worth quantifying in people rather than decimals. Applied to the 31,781 diabetic records in the training set, LightGBM's recall of 0.772 corresponds to roughly 24,519 of them correctly identified, against about 22,530 for Gaussian Naive Bayes — a difference of some 1,989 people, or 6.3 percentage points of the diabetic population. The four models are not interchangeable, even though their ROC-AUC values all fall within a range of 0.047.

8.3 Why the Performance Differed

The ranking is not arbitrary. It follows directly from how much structure each algorithm is able to represent, and the ordering matches what the data's own properties would predict.

- **Gaussian Naive Bayes finished last because its core assumption is false here.** The algorithm assumes every feature is independent of the others once the class is known. Section 2.5 showed

that is plainly untrue: general health correlates with poor physical-health days at 0.52, difficulty walking with the same variable at 0.48, and education with income at 0.45. When features carry overlapping information, Naive Bayes counts that evidence repeatedly and becomes overconfident. It also models every feature with a Gaussian curve, which is a poor description of fourteen columns that only ever take the values 0 and 1.

- **The Decision Tree finished third because a single tree must commit to hard splits.** It can represent interactions, which is why it beats Naive Bayes, but every decision is a yes/no cut on one variable at a time and every record in the same leaf receives an identical probability. With the depth capped at six to prevent memorisation, the whole of a continuous variable like BMI is reduced to a handful of bands. Its instability shows in the numbers: its recall varied across folds with a standard deviation of 0.0177, roughly four times LightGBM's.
- **Logistic Regression finished a close second because the relationships here are largely additive.** It cannot represent any interaction that is not specified in advance, which should be a serious handicap. That it is beaten by only 0.0067 of ROC-AUC tells us something real about the data: most of the available signal is carried by features acting independently and additively, so a weighted sum captures nearly all of it. Its smooth continuous treatment of BMI, age and general health is also an advantage over the tree's coarse bands, which is why it outperforms the Decision Tree.
- **LightGBM finished first because it combines both advantages.** It builds 400 shallow trees in sequence, each correcting the errors of those before it, so it captures interactions like the tree while its many small additive corrections approximate smooth relationships like the linear model. Its margin is small precisely because Section 2.5 found no strong individual correlations and no dominant interaction for it to exploit — there was limited extra structure available to find.

The general pattern is that performance tracks how well each algorithm's assumptions match the data. The dataset rewards additive combination of many weak signals, which is why the additive models lead; and it punishes the independence assumption, which is why Naive Bayes trails.

8.4 Strengths and Limitations of Each Model

Score alone is not the whole picture. Each model brings a different balance of accuracy, interpretability and cost, and the table below records that balance for all four.

Model	Strengths	Limitations
Logistic Regression	Fully interpretable — every coefficient is an odds ratio, so any individual prediction can be explained. Fast to train (0.46 s). Outputs well-behaved probabilities. Very stable across folds. Only 0.0067 ROC-AUC behind the winner.	Cannot represent interactions unless they are specified by hand. Assumes each predictor has a constant, monotonic effect on the log-odds, which is unlikely to be exactly true for age or BMI. Sensitive to feature scaling, so it depends on the preprocessing being correct.
Decision Tree	The most immediately readable of the four — the model is a flowchart a clinician can follow without any statistics. Captures interactions automatically. Needs no feature scaling. Fastest tree-based option (0.31 s).	Least stable of the four (recall SD 0.0177, four times LightGBM's), because small changes in the data can change a split and alter the tree below it. Splits continuous variables into coarse bands. Prone to memorising the data unless depth is capped, which then limits what it can express.
Gaussian Naive Bayes	By far the fastest to train (0.10 s). Needs very little data to estimate its parameters. Has no hyperparameters to tune. Provides an honest floor against which to judge whether the other models	Its independence assumption is clearly violated by this dataset, which is the direct cause of it finishing last. Models binary 0/1 columns with a Gaussian curve, which does not describe them. Produces poorly calibrated, overconfident

	earn their complexity.	probabilities. Lowest score on all seven metrics.
LightGBM	Best score on five of seven metrics and on every metric that is meaningful under imbalance. Most stable results (recall SD 0.0045). Captures interactions without being told about them. Handles imbalance directly via scale_pos_weight. Needs no feature scaling. Still trains in only 1.41 s.	Not directly interpretable — 400 trees cannot be drawn, so explanation depends on secondary tools such as the permutation importance in Section 7.5. The most hyperparameters to tune. Highest training cost of the four. Its advantage over a far simpler model is under one percent of ROC-AUC, so the added complexity buys comparatively little here.

Read together, the table explains why the choice made in Section 9 is a genuine trade rather than an obvious win. LightGBM is the better predictor; Logistic Regression is the more explainable model and is very nearly as accurate. Sections 9.2 and 9.3 set out why the predictive advantage was given priority for this particular application, and under what circumstances the opposite decision would be correct.

9. Selection of the Best Model

On the evidence of Sections 7 and 8, LightGBM is selected as the best model for this problem. The decision was not close on the metrics that matter, but neither was it made on a single number, and the reasoning is set out in full below.

9.1 The Decision

Criterion	Verdict
Selected model	LightGBM (gradient-boosted decision tree ensemble)
Runner-up	Logistic Regression
Basis of the decision	Highest score on six of the seven metrics measured, including every metric that is meaningful under class imbalance
Margin over the runner-up	0.0067 ROC-AUC, 0.0204 PR-AUC, 0.0222 recall

9.2 Reasoning

Five considerations support the choice.

- **It leads on every metric that matters here.** LightGBM records the highest recall (0.772), F1 (0.478), ROC-AUC (0.809), PR-AUC (0.460) and balanced accuracy (0.733). Logistic Regression edges it only on raw precision, by 0.003, and on raw accuracy, which Section 7.2 explains is misleading on imbalanced data. No competing model wins on any measure that would change the decision.
- **It recovers the most diabetic cases.** Recall is the metric of first importance for a screening tool, because a missed diabetic case may go undiagnosed for years whereas a false alarm costs only an unnecessary blood test. LightGBM finds 77.2% of diabetic respondents, against 74.9% for the nearest competitor. Across the 45,957 test records that difference is roughly 180 additional people correctly identified.
- **Its lead on PR-AUC is the most meaningful margin.** Precision-recall AUC is the summary least distorted by class imbalance, and it is where LightGBM's advantage is widest, at 0.0204

over Logistic Regression. That the gap is largest on the most imbalance-robust metric is evidence the advantage is real rather than an artefact of the scoring.

- **Its results are the most stable.** Across the five cross-validation folds, LightGBM's recall varied by a standard deviation of only 0.0045, the lowest of the four candidates and roughly a quarter of the Decision Tree's 0.0177. A model whose performance barely changes with the sample it is trained on is the safer choice.
- **It handles feature interactions the linear model cannot.** Section 2.5 showed that no individual feature correlates strongly with diabetes; the highest was general health at 0.30. Predictive power here therefore comes from combining weak signals, which is precisely what a boosted ensemble does and what a linear model cannot do unless the interactions are specified in advance.

9.3 The Argument Against, and Why It Was Rejected

There is a real case for Logistic Regression, and it deserves to be stated rather than glossed over. It trails LightGBM by less than one percent of ROC-AUC while being incomparably easier to explain: it is a single equation, its coefficients convert directly into the odds ratios shown in Figure 9, and a clinician can see exactly why any given person was flagged. In a deployed clinical system subject to regulatory scrutiny, that transparency could well outweigh a fractional gain in accuracy.

It was not selected here for two reasons. First, the objective set out in Section 1.2 is to identify as many at-risk individuals as possible for follow-up testing, and on that objective LightGBM is measurably better. Second, the interpretability gap is narrower than it appears: the permutation importance analysis in Section 7.5 already explains which features drive LightGBM's decisions, and its ranking agrees with both the correlation analysis and the Decision Tree. The model is not the black box it is often assumed to be.

That said, the closeness of the two is a genuine finding rather than an inconvenience, and it is recorded here as such. Logistic Regression remains the recommended fallback if this work were ever taken towards deployment in a setting where every decision must be individually justified.

10. Testing: Final Evaluation on the Held-Out Set

With a single model chosen, the 45,957 held-out records described in Section 3.6 were used for the first and only time. Every figure in this section comes from data that played no part in training the model, in tuning it, or in selecting it over its three competitors, and is therefore the most honest estimate available of how the model would behave on new respondents.

10.1 Headline Results

Metric	Cross-validation (training set)	Test set (held out)	Difference
ROC-AUC	0.8088	0.8142	+0.0054
PR-AUC	0.4600	0.4712	+0.0112
Recall	0.7715	0.7728	+0.0013
Precision	0.3458	0.3486	+0.0028
F1	0.4776	0.4805	+0.0029
Accuracy	0.7082	0.7111	+0.0029

Balanced accuracy	0.7332	0.7355	+0.0023
-------------------	--------	--------	---------

The test scores are marginally higher than the cross-validated estimates across every metric, with the largest difference being 0.011. This is the result one hopes for: had the test figures fallen well short of the cross-validated ones, it would indicate the model had been over-fitted to the training data or that the comparison had somehow leaked information from the test set. Differences this small in the favourable direction are ordinary sampling variation and confirm that the preprocessing discipline described in Section 3.7 held.

10.2 Confusion Matrix for the Selected Model

A single score conceals the thing that matters most in a medical application, which is how many people the model missed. The confusion matrix below makes that visible.

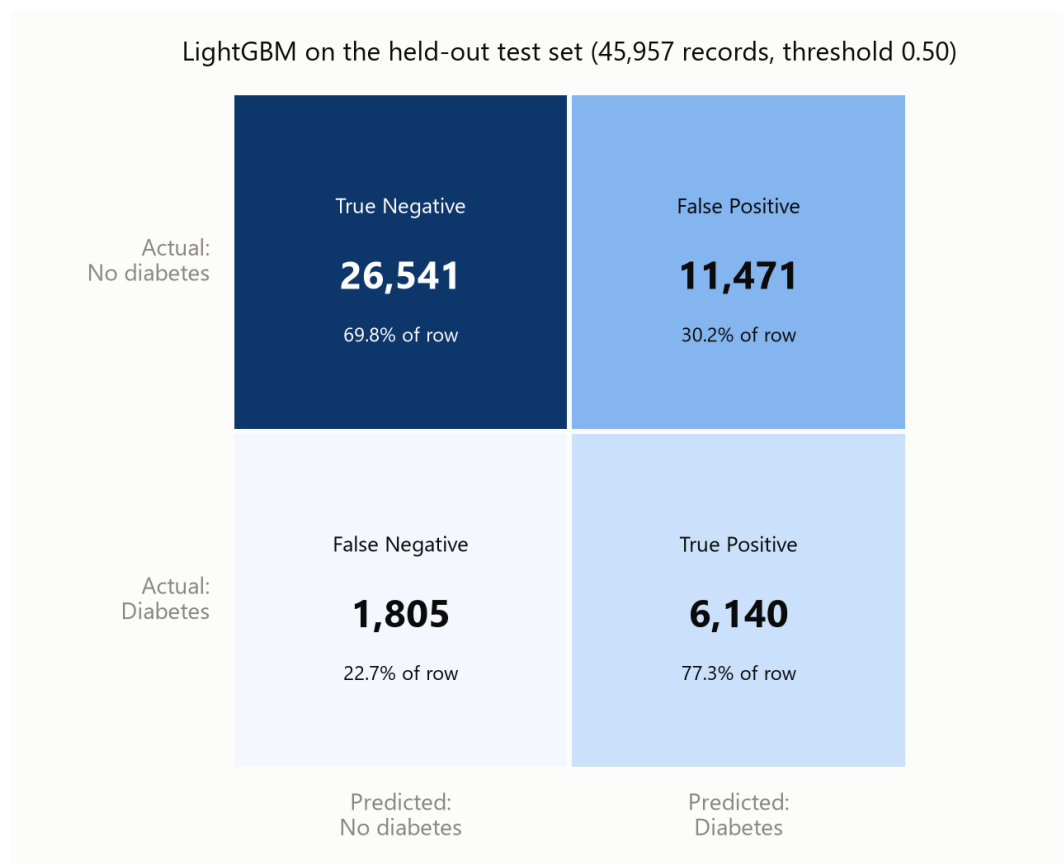


Figure 12: Confusion matrix for LightGBM on the held-out test set at the default decision threshold of 0.50.

Of the 7,945 genuinely diabetic respondents in the test set, the model correctly identified 6,140 and missed 1,805. Of the 38,012 non-diabetic respondents, it correctly cleared 26,541 and incorrectly flagged 11,471. In the terms of the intended application, screening this population of 45,957 people would refer 17,611 of them for a confirmatory blood test, and that referral list would contain roughly three quarters of everyone who actually has the condition.

10.3 The Decision Threshold Is a Policy Choice

All the figures above use the conventional cut-off of 0.50: anyone the model scores above 0.50 is flagged. There is nothing sacred about that number, and moving it trades the two kinds of error against one another. Figure 13 shows the whole trade-off.

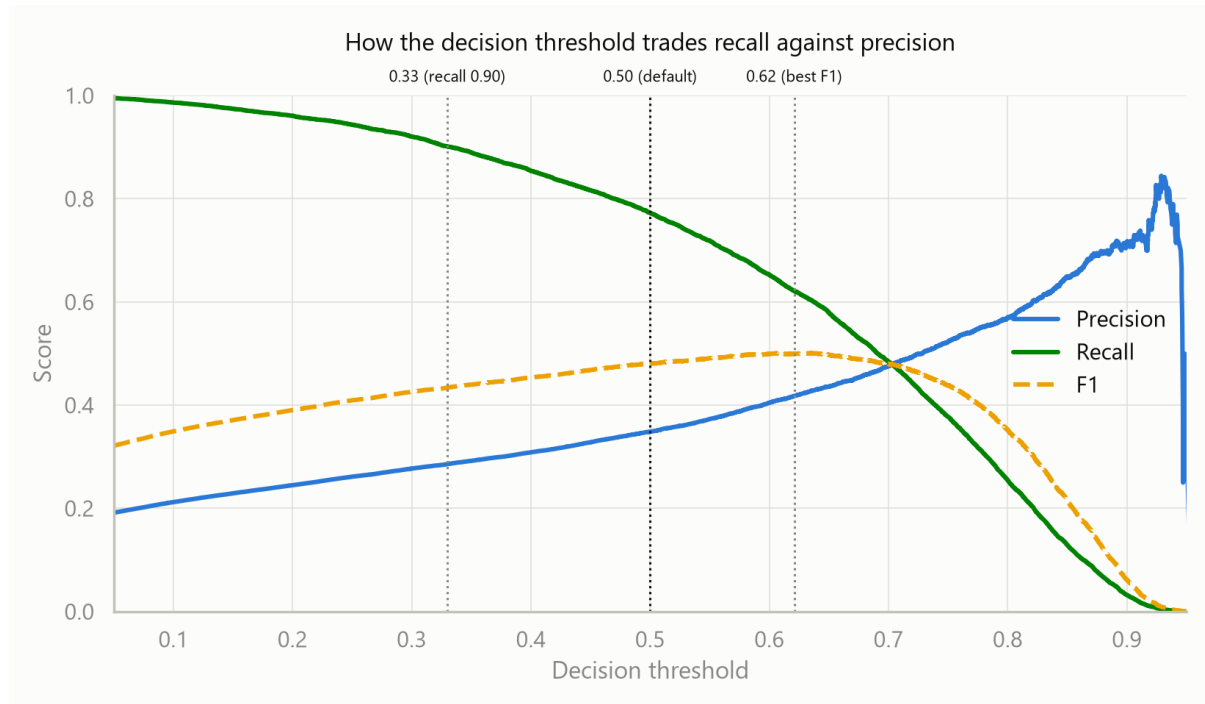


Figure 13: Precision, recall and F1 across the full range of decision thresholds, with three candidate operating points marked.

Three operating points were examined. The thresholds themselves were chosen using out-of-fold predictions on the training set, not on the test set, so the comparison below remains honest.

Threshold	Accuracy	Precision	Recall	F1	Diabetics missed	Healthy people flagged
0.33 (high recall)	0.594	0.286	0.902	0.434	782	17,896
0.50 (default)	0.711	0.349	0.773	0.481	1,805	11,471
0.62 (best F1)	0.785	0.418	0.621	0.500	3,009	6,880

The table shows why accuracy is such a poor guide on this problem. Raising the threshold to 0.62 produces the best F1 score and lifts accuracy from 0.711 to 0.785, which looks like an improvement, yet it misses 3,009 diabetic people instead of 1,805. The threshold that scores best on the summary statistics is the one that fails worst at the actual task.

The default of 0.50 is retained as the recommended operating point. It catches 77.3% of cases at a cost of 11,471 unnecessary blood tests, which is a reasonable balance when the follow-up test is cheap and non-invasive. Where missing a case is judged unacceptable, the 0.33 threshold reduces missed diagnoses to 782, catching 90.2% of cases, at the price of flagging almost half the population. That decision belongs to whoever operates the screening programme, not to the model.

10.4 Conclusion

The project set out to determine whether diabetes risk can be predicted from self-reported health and lifestyle information alone, without any laboratory measurement. The answer is a qualified yes. LightGBM achieves a ROC-AUC of 0.814 on data it has never seen, identifying 77.3% of diabetic respondents from twenty-one survey answers that cost nothing to collect.

The qualification is precision. Roughly two in three flagged individuals do not have diabetes, so the output is a referral list rather than a diagnosis. That is an appropriate role for a screening instrument, and the model would be genuinely useful for deciding whom to test first in a population where testing everybody is not affordable. It would not be appropriate as a substitute for an HbA1c or fasting glucose test, and nothing in these results suggests otherwise.

11. References

- [1] Centers for Disease Control and Prevention. Behavioral Risk Factor Surveillance System Survey Data, 2015. U.S. Department of Health and Human Services, Atlanta, Georgia. Available at: https://www.cdc.gov/brfss/annual_data/annual_2015.html
- [2] Teboul, A. Diabetes Health Indicators Dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset>
- [3] Xie, Z., Nikolayeva, O., Luo, J. and Li, D. (2019). Building Risk Prediction Models for Type 2 Diabetes Using Machine Learning Techniques. *Preventing Chronic Disease*, 16, E130.
- [4] Centers for Disease Control and Prevention (2020). National Diabetes Statistics Report. U.S. Department of Health and Human Services, Atlanta, Georgia.
- [5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [6] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

Appendix A — Complete Code

The complete source code for the project, reproduced exactly as it was run. The work is split across three Jupyter notebooks that execute in sequence, each writing files the next one reads. Running them in order reproduces every number and every figure in this report.

Only the code cells are reproduced here. The notebooks themselves, complete with their commentary and saved output, are included with the submission in the 02_Code folder.

Notebook	Purpose	Report sections
01_data_exploration_and_preprocessing.ipynb	Loads and explores the data, cleans it, and produces the train/test split	2, 3, 4
02_model_training_and_selection.ipynb	Trains the four models and compares them by cross-validation	5, 6, 7, 8
03_final_evaluation.ipynb	Evaluates the selected model once on the held-out test set	9, 10

A.1 Data Exploration and Preprocessing

Source file: 01_data_exploration_and_preprocessing.ipynb

Cell 1

```
import os
import glob

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
%matplotlib inline

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)

print("pandas:", pd.__version__)
```

Cell 2

```
CSV_NAME = "diabetes_012_health_indicators_BRFSS2015.csv"

def load_dataset():
    """Load the 012 CSV locally if present, else download via kagglehub."""
    if os.path.exists(CSV_NAME):
        print("Loading local file:", CSV_NAME)
        return pd.read_csv(CSV_NAME)

    import kagglehub
    print("No local CSV found. Downloading via kagglehub...")
    path = kagglehub.dataset_download("alexteboul/diabetes-health-indicators-dataset")

    matches = glob.glob(os.path.join(path, "**", CSV_NAME), recursive=True)
    if not matches: # fall back to any csv with '012' in the name
        matches = [f for f in glob.glob(os.path.join(path, "**", "*.csv"), recursive=True)]
```

```

        if "012" in os.path.basename(f)]
    if not matches:
        raise FileNotFoundError(f"Could not find {CSV_NAME} in {path}")

    print("Loading:", matches[0])
    return pd.read_csv(matches[0])

```

```

df = load_dataset()
df.head()

```

Cell 3

```

rows, cols = df.shape
print(f"Rows      : {rows:,}")
print(f"Columns   : {cols}")
print(f"Memory    : {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
print(f"\nColumns:", list(df.columns))

```

Cell 4

```

df.info()

```

Cell 5

```

summary = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing": df.isnull().sum(),
    "unique": df.nunique(),
    "min": df.min(numeric_only=True),
    "max": df.max(numeric_only=True),
})
print(f"Exact duplicate rows: {df.duplicated().sum():,}"
      f" ({df.duplicated().mean()*100:.1f}%)\n")
summary

```

Cell 6

```

df.describe().T

```

Cell 7

```

target = "Diabetes_012"
labels = {0: "No diabetes", 1: "Prediabetes", 2: "Diabetes"}

counts = df[target].value_counts().sort_index()
pcts = df[target].value_counts(normalize=True).sort_index() * 100
balance = pd.DataFrame({"count": counts, "percent": pcts.round(2)})
balance.index = [labels[i] for i in balance.index]
print(balance)
print(f"\nAlways predicting 'No diabetes' would score {pcts.iloc[0]:.1f}% accuracy.")

plt.figure(figsize=(6, 4))
ax = sns.barplot(x=[labels[i] for i in counts.index], y=counts.values,
                 hue=[labels[i] for i in counts.index], palette="Set2", legend=False)
for i, v in enumerate(counts.values):
    ax.text(i, v, f"{v:,}", ha="center", va="bottom")
plt.title("Class balance (Diabetes_012)")
plt.ylabel("count")
plt.show()

```

Cell 8

```

corr = df.corr(numeric_only=True)

plt.figure(figsize=(15, 13))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            square=True, linewidths=0.5, annot_kws={"size": 7},
            cbar_kws={"shrink": 0.8})
plt.title("Correlation heatmap – BRFSS 2015 diabetes indicators")
plt.tight_layout()
plt.savefig("brfss_correlation_heatmap.png", dpi=150)
plt.show()
print("Saved to brfss_correlation_heatmap.png")

```

Cell 9

```

target_corr = corr[target].drop(target).sort_values(ascending=False)

plt.figure(figsize=(8, 8))
sns.barplot(x=target_corr.values, y=target_corr.index, hue=target_corr.index,
            palette="coolwarm_r", legend=False)
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Correlation of each feature with Diabetes_012")
plt.xlabel("Pearson correlation")
plt.tight_layout()
plt.show()

target_corr.round(3)

```

Cell 10

```

df["is_diabetic"] = (df[target] == 2).astype(int)

def prevalence_by(col):
    g = df.groupby(col)["is_diabetic"].mean() * 100
    return g.round(1)

fig, axes = plt.subplots(2, 2, figsize=(13, 9))

# General health (1=excellent ... 5=poor)
prevalence_by("GenHlth").plot(kind="bar", ax=axes[0, 0], color="#d1495b")
axes[0, 0].set_title("Diabetes % by General Health (1=excellent, 5=poor)")
axes[0, 0].set_ylabel("% diabetic")

# High blood pressure
prevalence_by("HighBP").plot(kind="bar", ax=axes[0, 1], color="#edae49")
axes[0, 1].set_title("Diabetes % by HighBP (0=no, 1=yes)")

# Age bucket (BRFSS 1..13)
prevalence_by("Age").plot(kind="bar", ax=axes[1, 0], color="#00798c")
axes[1, 0].set_title("Diabetes % by Age bucket (1=18-24 ... 13=80+)")
axes[1, 0].set_ylabel("% diabetic")

# Difficulty walking
prevalence_by("DiffWalk").plot(kind="bar", ax=axes[1, 1], color="#66a182")
axes[1, 1].set_title("Diabetes % by DiffWalk (0=no, 1=yes)")

plt.tight_layout()
plt.savefig("brfss_prevalence.png", dpi=150)
plt.show()

```

Cell 11

```

plt.figure(figsize=(9, 5))
sns.kdeplot(data=df, x="BMI", hue=df[target].map(labels),

```

```

        fill=True, common_norm=False, palette="Set2", clip=(10, 60))
plt.title("BMI distribution by diabetes status")
plt.xlim(10, 60)
plt.show()

print(df.groupby(target)["BMI"].median().rename(labels).round(1))

```

Cell 12

```

# Fresh copy so the exploration above stays intact
data = df.copy()

# Remove the helper column added in Section 9 (it encodes the target -> would leak)
data = data.drop(columns=[c for c in ["is_diabetic"] if c in data.columns])

# 1) Drop exact duplicate rows
before = len(data)
data = data.drop_duplicates().reset_index(drop=True)
print(f"Duplicates removed: {before - len(data):,} "
      f"f"((before - len(data)) / before * 100:.1f)%") -> {len(data):,} rows left")

# 2) Confirm there is nothing to impute
print("Missing values total:", int(data.isnull().sum().sum()))

# 3) Clip implausible BMI extremes (raw max is ~98) into a sensible clinical range
n_clipped = ((data["BMI"] < 12) | (data["BMI"] > 60)).sum()
data["BMI"] = data["BMI"].clip(lower=12, upper=60)
print(f"BMI values clipped to [12, 60]: {n_clipped:,}")

```

Cell 13

```

# 1 = prediabetes OR diabetes, 0 = none.
# (To instead predict ONLY full diabetes, use: (data["Diabetes_012"] == 2))
data["Diabetes_binary"] = (data["Diabetes_012"] >= 1).astype(int)
data = data.drop(columns=["Diabetes_012"])

counts = data["Diabetes_binary"].value_counts().sort_index()
print(counts)
print(f"\nPositive (at-risk) rate: {data['Diabetes_binary'].mean() * 100:.1f}%")

```

Cell 14

```

TARGET = "Diabetes_binary"

binary_cols = [
    "HighBP", "HighChol", "CholCheck", "Smoker", "Stroke",
    "HeartDiseaseorAttack", "PhysActivity", "Fruits", "Veggies",
    "HvyAlcoholConsump", "AnyHealthcare", "NoDocbcCost", "DiffWalk", "Sex",
]
scale_cols = ["BMI", "MentHlth", "PhysHlth", "GenHlth", "Age", "Education", "Income"]

# Fail loudly if a column was missed or mislabelled
missing = set(data.columns) ^ set(binary_cols + scale_cols + [TARGET])
assert not missing, f"Unclassified columns: {missing}"

print(f"Binary (passthrough): {len(binary_cols)}")
print(f"Scaled (standardized): {len(scale_cols)}")

```

Cell 15

```

from sklearn.model_selection import train_test_split

```

```

X = data.drop(columns=[TARGET])
y = data[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

print("X_train:", X_train.shape, " X_test:", X_test.shape)
print("Positive rate -> train: {:.1f}% test: {:.1f}%".format(
    y_train.mean() * 100, y_test.mean() * 100))

```

Cell 16

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), scale_cols)],
    remainder="passthrough", # binary 0/1 columns untouched
    verbose_feature_names_out=False,
)

X_train_proc = preprocessor.fit_transform(X_train) # fit + transform on TRAIN
X_test_proc = preprocessor.transform(X_test) # transform only on TEST

feature_names = preprocessor.get_feature_names_out()
print("Processed train matrix:", X_train_proc.shape)

# Verify the scaled columns are now ~mean 0, std 1 on the training set
check = (pd.DataFrame(X_train_proc, columns=feature_names)[scale_cols]
        .agg(["mean", "std"]).round(3))
print(check)

```

Cell 17

```

import numpy as np
import joblib

np.savez_compressed(
    "brfss_train_test.npz",
    X_train=X_train_proc, X_test=X_test_proc,
    y_train=y_train.to_numpy(), y_test=y_test.to_numpy(),
    feature_names=np.array(feature_names, dtype=object),
)
joblib.dump(preprocessor, "brfss_preprocessor.joblib")

print("Saved:")
print(" brfss_train_test.npz - processed X/y arrays + feature names")
print(" brfss_preprocessor.joblib - fitted scaler (reuse on any new raw data)")

```

A.2 Model Training and Selection

Source file: *02_model_training_and_selection.ipynb*

Cell 1

```

import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```

```

import joblib

from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
from lightgbm import LGBMClassifier

sns.set_theme(style="whitegrid")
%matplotlib inline
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)

print("Libraries loaded.")

```

Cell 2

```

data = np.load("brfss_train_test.npz", allow_pickle=True)

X_train = data["X_train"]
X_test = data["X_test"]      # loaded for completeness only - NOT used in this notebook
y_train = data["y_train"]
y_test = data["y_test"]      # likewise
feature_names = list(data["feature_names"])

print(f"Training set : {X_train.shape[0]:,} rows x {X_train.shape[1]} features")
print(f"Testing set  : {X_test.shape[0]:,} rows (held back, not used below)")
print(f"\nFeatures: {feature_names}")

```

Cell 3

```

# Class balance in the training set, and the weight needed to correct for it
n_pos = int(y_train.sum())
n_neg = int((y_train == 0).sum())
scale_pos_weight = n_neg / n_pos

print(f"Non-diabetic (0): {n_neg:,}")
print(f"Diabetic      (1): {n_pos:,}")
print(f"Positive rate   : {y_train.mean() * 100:.2f}%")
print(f"\nscale_pos_weight = {n_neg:,} / {n_pos:,} = {scale_pos_weight:.4f}")
print("\nEvery model below is told to weight the diabetic class by roughly this much,")
print("so that the minority class is not ignored during training.")

```

Cell 4

```

models = {
    "Logistic Regression": LogisticRegression(
        class_weight="balanced",  # corrects the 17% / 83% imbalance
        max_iter=1000,           # the default 100 is not always enough to converge
        random_state=42,
    ),

    "Decision Tree": DecisionTreeClassifier(
        max_depth=6,             # unconstrained, a tree on 183k rows memorises the data
        min_samples_leaf=50,      # no rule may be based on fewer than 50 people
        class_weight="balanced",
        random_state=42,
    ),

    "Gaussian Naive Bayes": GaussianNB(
        priors=[0.5, 0.5],       # NB has no class_weight; equal priors is the equivalent
    ),
}

```



```

"LightGBM": LGBMClassifier(
    n_estimators=400,          # number of boosting rounds
    learning_rate=0.05,       # smaller steps, more rounds - the usual trade
    num_leaves=31,
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    n_jobs=4,
    verbose=-1,               # silence LightGBM's per-round chatter
),
}

for name, m in models.items():
    print(f"{name:22s} -> {type(m).__name__}")

```

Cell 5

```

scoring = ["recall", "precision", "f1", "roc_auc", "average_precision"]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rows = []
for name, model in models.items():
    t0 = time.time()
    res = cross_validate(model, X_train, y_train, cv=cv, scoring=scoring, n_jobs=5)
    elapsed = time.time() - t0

    rows.append({
        "Model": name,
        "Recall": res["test_recall"].mean(),
        "Precision": res["test_precision"].mean(),
        "F1": res["test_f1"].mean(),
        "ROC-AUC": res["test_roc_auc"].mean(),
        "PR-AUC": res["test_average_precision"].mean(),
        "Recall SD": res["test_recall"].std(),
        "Fit time (s)": res["fit_time"].mean(),
    })
    print(f"{name:22s} finished in {elapsed:5.1f}s")

results = pd.DataFrame(rows).set_index("Model")
print("\nAll models cross-validated.")

```

Cell 6

```

# Sorted by ROC-AUC, the fairest single summary for comparing models
results.sort_values("ROC-AUC", ascending=False).round(4)

```

Cell 7

```

PALETTE = ["#2a78d6", "#008300", "#e87ba4", "#eda100"] # one fixed colour per model
colours = {name: PALETTE[i] for i, name in enumerate(models)}

metrics = ["Recall", "Precision", "F1", "ROC-AUC", "PR-AUC"]
fig, axes = plt.subplots(1, len(metrics), figsize=(16, 4.5), sharey=False)

for ax, metric in zip(axes, metrics):
    vals = results[metric].sort_values(ascending=False)
    bars = ax.bar(range(len(vals)), vals.values,
                  color=[colours[n] for n in vals.index], width=0.65)
    for b, v in zip(bars, vals.values):
        ax.text(b.get_x() + b.get_width() / 2, v + 0.012, f"{v:.3f}",
                ha="center", va="bottom", fontsize=9)
    ax.set_title(metric, fontsize=11)
    ax.set_xticks(range(len(vals)))
    ax.set_xticklabels([n.replace(" ", "\n") for n in vals.index], fontsize=8)
    ax.set_ylim(0, max(vals.values) * 1.22)

```

```

ax.spines[["top", "right"]].set_visible(False)

fig.suptitle("Cross-validated performance of the four candidate models", fontsize=13)
plt.tight_layout(rect=[0, 0, 1, 0.94])
plt.savefig("model_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

print("Saved to model_comparison.png")

```

Cell 8

```

best = results["ROC-AUC"].idxmax()
simple_best = results.drop(index="LightGBM")["ROC-AUC"].idxmax()
gap = results.loc[best, "ROC-AUC"] - results.loc[simple_best, "ROC-AUC"]

print(f"Best overall          : {best} (ROC-AUC {results.loc[best, 'ROC-AUC']:.4f})")
print(f"Best simple model     : {simple_best} (ROC-AUC {results.loc[simple_best, 'ROC-AUC']:.4f})")
print(f"Gap between the two    : {gap:.4f}")
print()
print("A small gap is a genuinely useful finding, not a disappointment: it means the")
print("simple, explainable model is doing almost as well as the advanced one, which is")
print("a strong argument in a medical setting where decisions must be justified.")

```

Cell 9

```

fitted = {}
for name, model in models.items():
    t0 = time.time()
    model.fit(X_train, y_train)
    fitted[name] = model
    fname = name.lower().replace(" ", "_") + ".joblib"
    joblib.dump(model, fname)
    print(f"{name:22s} trained in {time.time() - t0:5.1f}s -> {fname}")

results.to_csv("model_comparison_results.csv")
print("\nScores saved to model_comparison_results.csv")

```

Cell 10

```

lr = fitted["Logistic Regression"]

odds = (pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": lr.coef_[0],
    "Odds ratio": np.exp(lr.coef_[0]),
}).sort_values("Odds ratio", ascending=False).reset_index(drop=True))

odds.round(3)

```

Cell 11

```

plot_df = odds.sort_values("Odds ratio")

plt.figure(figsize=(8, 7))
cols = ["#2a78d6" if v < 1 else "#d03b3b" for v in plot_df["Odds ratio"]]
plt.barh(plot_df["Feature"], plot_df["Odds ratio"], color=cols, height=0.68)
plt.axvline(1.0, color="black", linewidth=1.2)
plt.xlabel("Odds ratio (1.0 = no effect)")
plt.title("Logistic Regression: effect of each feature on the odds of diabetes")
for i, v in enumerate(plot_df["Odds ratio"]):
    plt.text(v + 0.02, i, f"{v:.2f}", va="center", fontsize=8)
plt.tight_layout()
plt.savefig("logreg_odds_ratios.png", dpi=150)

```

```
plt.show()
```

Cell 12

```
lgbm = fitted["LightGBM"]

imp = (pd.DataFrame({
    "Feature": feature_names,
    "Importance": lgbm.feature_importances_,
}).sort_values("Importance", ascending=True).reset_index(drop=True))

plt.figure(figsize=(8, 7))
plt.barh(imp["Feature"], imp["Importance"], color="#2a78d6", height=0.68)
plt.xlabel("Number of times used as a split")
plt.title("LightGBM: feature importance")
plt.tight_layout()
plt.savefig("lightgbm_importance.png", dpi=150)
plt.show()

imp.sort_values("Importance", ascending=False).reset_index(drop=True)
```

Cell 13

```
plt.figure(figsize=(20, 9))
plot_tree(
    fitted["Decision Tree"],
    max_depth=3,
    feature_names=feature_names,
    class_names=["No diabetes", "Diabetes"],
    filled=True,
    rounded=True,
    fontsize=8,
    impurity=False,
)
plt.title("Decision Tree (top 3 levels of 6)", fontsize=14)
plt.tight_layout()
plt.savefig("decision_tree.png", dpi=150, bbox_inches="tight")
plt.show()
```

A.3 Final Evaluation

Source file: 03_final_evaluation.ipynb

Cell 1

```
import json

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import (accuracy_score, balanced_accuracy_score,
                             precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score,
                             confusion_matrix, precision_recall_curve,
                             classification_report)

from lightgbm import LGBMClassifier

sns.set_theme(style="whitegrid")
%matplotlib inline
```

```
pd.set_option("display.max_columns", None)
```

```
print("Libraries loaded.")
```

Cell 2

```
data = np.load("brfss_train_test.npz", allow_pickle=True)
X_train, y_train = data["X_train"], data["y_train"]
X_test, y_test = data["X_test"], data["y_test"]
feature_names = list(data["feature_names"])

scale_pos_weight = (y_train == 0).sum() / y_train.sum()

print(f"Training : {X_train.shape[0]:,} rows")
print(f"Testing : {X_test.shape[0]:,} rows ({int(y_test.sum()):,} diabetic, "
      f"{int((y_test == 0).sum()):,} non-diabetic)")
```

Cell 3

```
best_model = LGBMClassifier(
    n_estimators=400, learning_rate=0.05, num_leaves=31,
    scale_pos_weight=scale_pos_weight, random_state=42, n_jobs=4, verbose=-1,
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof_proba = cross_val_predict(best_model, X_train, y_train, cv=cv,
                              method="predict_proba", n_jobs=5)[: , 1]

prec, rec, thr = precision_recall_curve(y_train, oof_proba)
f1_curve = 2 * prec * rec / np.clip(prec + rec, 1e-12, None)

i_f1 = int(np.nanargmax(f1_curve[:-1]))
thr_f1 = float(thr[i_f1])

i_r90 = np.where(rec[:-1] >= 0.90)[0][-1]
thr_r90 = float(thr[i_r90])

print(f"Best-F1 threshold      : {thr_f1:.3f}")
print(f"Threshold for recall 0.90: {thr_r90:.3f}")
print(f"Default                : 0.500")
```

Cell 4

```
final_model = LGBMClassifier(
    n_estimators=400, learning_rate=0.05, num_leaves=31,
    scale_pos_weight=scale_pos_weight, random_state=42, n_jobs=4, verbose=-1,
)
final_model.fit(X_train, y_train)
joblib.dump(final_model, "final_model_lightgbm.joblib")

test_proba = final_model.predict_proba(X_test)[: , 1]

print(f"Test ROC-AUC : {roc_auc_score(y_test, test_proba):.4f}")
print(f"Test PR-AUC : {average_precision_score(y_test, test_proba):.4f}")
```

Cell 5

```
def evaluate(threshold, label):
    pred = (test_proba >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    return {
        "Operating point": label,
```

```

        "Threshold": round(threshold, 3),
        "Accuracy": round(accuracy_score(y_test, pred), 4),
        "Balanced acc": round(balanced_accuracy_score(y_test, pred), 4),
        "Precision": round(precision_score(y_test, pred), 4),
        "Recall": round(recall_score(y_test, pred), 4),
        "F1": round(f1_score(y_test, pred), 4),
        "Specificity": round(tn / (tn + fp), 4),
        "Missed cases (FN)": int(fn),
        "False alarms (FP)": int(fp),
    }

summary = pd.DataFrame([
    evaluate(thr_r90, "High recall"),
    evaluate(0.50, "Default (recommended)"),
    evaluate(thr_f1, "Best F1"),
]).set_index("Operating point")

summary

```

Cell 6

```

y_pred = (test_proba >= 0.50).astype(int)
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, square=True,
            linewidths=2, annot_kws={"size": 15, "weight": "bold"},
            xticklabels=["Predicted:\nNo diabetes", "Predicted:\nDiabetes"],
            yticklabels=["Actual:\nNo diabetes", "Actual:\nDiabetes"], ax=ax)
ax.set_title("LightGBM on the held-out test set (threshold 0.50)", fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("confusion_matrix_test.png", dpi=150)
plt.show()

print(f"Diabetic people found : {tp:,} of {tp + fn:,} ({tp / (tp + fn) * 100:.1f}%)")
print(f"Diabetic people missed : {fn:,}")
print(f"Healthy people flagged : {fp:,} of {tn + fp:,}")
print(f"Total referred to test : {tp + fp:,}")

```

Cell 7

```

print(classification_report(y_test, y_pred,
                           target_names=["No diabetes", "Diabetes"], digits=4))

```

Cell 8

```

p_t, r_t, t_t = precision_recall_curve(y_test, test_proba)
f1_t = 2 * p_t * r_t / np.clip(p_t + r_t, 1e-12, None)

plt.figure(figsize=(8.5, 5))
plt.plot(t_t, p_t[:-1], label="Precision", linewidth=2)
plt.plot(t_t, r_t[:-1], label="Recall", linewidth=2)
plt.plot(t_t, f1_t[:-1], label="F1", linewidth=2, linestyle="--")
for t, lab in [(thr_r90, "recall 0.90"), (0.5, "default"), (thr_f1, "best F1")]:
    plt.axvline(t, color="grey", linestyle=":", linewidth=1)
    plt.text(t, 1.01, f"{t:.2f}\n{lab}", ha="center", fontsize=8)
plt.xlabel("Decision threshold")
plt.ylabel("Score")
plt.title("How the decision threshold trades recall against precision", pad=26)
plt.xlim(0.05, 0.95)
plt.ylim(0, 1)

```

```

plt.legend(loc="center right")
plt.tight_layout()
plt.savefig("threshold_tradeoff.png", dpi=150)
plt.show()

```

Cell 9

```

cv_scores = pd.read_csv("model_comparison_full.csv", index_col=0).loc["LightGBM"]
test_row = evaluate(0.50, "test")

comparison = pd.DataFrame({
    "Cross-validation": [cv_scores["ROC-AUC"], cv_scores["PR-AUC"],
                        cv_scores["Recall"], cv_scores["Precision"],
                        cv_scores["F1"], cv_scores["Accuracy"]],
    "Test set": [roc_auc_score(y_test, test_proba),
                average_precision_score(y_test, test_proba),
                test_row["Recall"], test_row["Precision"],
                test_row["F1"], test_row["Accuracy"]],
}, index=["ROC-AUC", "PR-AUC", "Recall", "Precision", "F1", "Accuracy"])
comparison["Difference"] = comparison["Test set"] - comparison["Cross-validation"]

comparison.round(4)

```

Cell 10

```

results = {
    "best_model": "LightGBM",
    "test_roc_auc": round(roc_auc_score(y_test, test_proba), 4),
    "test_pr_auc": round(average_precision_score(y_test, test_proba), 4),
    "operating_points": summary.reset_index().to_dict(orient="records"),
    "confusion_matrix_at_0.50": {"TN": int(tn), "FP": int(fp),
                                "FN": int(fn), "TP": int(tp)},
}
with open("final_test_results.json", "w") as f:
    json.dump(results, f, indent=1)

print("Saved final_test_results.json and final_model_lightgbm.joblib")

```